

Combining GP interactions and police call records

```
In [1]: import warnings
warnings.filterwarnings("ignore")
```

```
In [2]: # from pandas_gbq import read_gbq
import matplotlib.pyplot as plt
import matplotlib.dates as mdates
import seaborn as sns
import numpy as np
from scipy.stats import zscore
import pandas as pd
import contextily as cx
```

```
In [3]: sns.set(rc={'figure.figsize':(11, 4)})
plt.rcParams['font.family'] = 'DejaVu Sans'

font_properties = {'family': 'sans-serif', 'weight': 'bold', 'size': 12}
tickfont_properties = {'family': 'sans-serif', 'size': 9}
```

```
In [4]: gp_lsoa_df = pd.read_csv('../data/GPinteractions_withLSOA.csv', index_col=0)
gp_lsoa_df.drop('index', axis=1, inplace=True)
```

The data is aggregated from `gp_lsoa_df` by grouping it based on `person_id_hashed` and `DateEventRecorded`. For each group, it calculates the number of unique LSOAs assigned on that date for that person. The result is stored in a new DataFrame `df_agg_lsoa_count`, with a column named `Count_lsoa` representing the count of distinct LSOAs per person per day. This is conducted to ensure there are no duplicate LSOAs in the data.

```
In [5]: df_agg_lsoa_count = gp_lsoa_df.groupby(['person_id_hashed', 'DateEventRecorded'])
df_agg_lsoa_count['Count_lsoa'].value_counts()
```

```
Out[5]: Count_lsoa
1      159811
0       19547
Name: count, dtype: int64
```

After removing all the null values from the data, we check for the number of unique LSOAs present in the data.

```
In [6]: df_mh_day_agg_lsoa_filter = gp_lsoa_df.dropna(subset='Assigned_LSOA')
unique_historic_lsoas = df_mh_day_agg_lsoa_filter['Assigned_LSOA'].unique()
len(unique_historic_lsoas)
```

```
Out[6]: 7571
```

Now, we will filter the data based on the LSOAs present in the Bradford to ensure we are considering only the interactions that occurred within the Bradford region.

```
In [7]: import geopandas as gpd

all_lsoa = gpd.read_file('../data/LSOA_2011_full.geojson')
df_lsoa_bradford = all_lsoa[all_lsoa.LSOA11NM.str.contains("Bradford")]
df_mh_day_agg_lsoa_bd = df_mh_day_agg_lsoa_filter[df_mh_day_agg_lsoa_filter.Assi
df_mh_day_agg_lsoa_bd.shape
```

```
Out[7]: (109513, 14)
```

```
In [8]: df_mh_day_agg_lsoa_bd['DateEventRecorded'] = pd.to_datetime(df_mh_day_agg_lsoa_bd
df_mh_day_agg_lsoa_bd['Month'] = df_mh_day_agg_lsoa_bd['DateEventRecorded'].dt.m
```

```
In [9]: df_monthly_counts_lsoa = df_mh_day_agg_lsoa_bd.groupby(['Assigned_LSOA', 'YearTr
df_monthly_counts_lsoa_pivot = df_monthly_counts_lsoa.pivot(index='Assigned_LSOA
```

```
In [10]: years_in_data = ['17-18', '18-19', '19-20', '20-21', '21-22']

for year in years_in_data:
    # Select all columns for that year
    cols_for_year = df_monthly_counts_lsoa_pivot.loc[:, df_monthly_counts_lsoa_p
    # Compute the sum across months for that year
    df_monthly_counts_lsoa_pivot[f'{year}_Total'] = cols_for_year.sum(axis=1)
```

```
In [11]: def convert_multiindex_to_date(col_tuple):
    year_str = col_tuple[0].split('-')[0]
    year = int('20' + year_str)
    month = col_tuple[1]
    if month in ['January', 'February', 'March']:
        year += 1
    return pd.to_datetime(f"{month}-{year}", format='%B-%Y')

# Apply to all column tuples
new_dates = [convert_multiindex_to_date(col) for col in df_monthly_counts_lsoa_p
df_monthly_counts = df_monthly_counts_lsoa_pivot.iloc[:, :-5]

df_monthly_counts.columns = sorted(new_dates)
```

```
In [ ]: import plotly.express as px
import plotly.graph_objects as go
import plotly.io as pio

pio.renderers.default = 'browser'
# Reset index to bring Assigned_LSOA into a column
df_long = df_monthly_counts.reset_index().melt(id_vars='Assigned_LSOA', var_name
total_counts = df_long.groupby('Assigned_LSOA')['Call Counts'].sum().reset_index
total_counts.columns = ['Assigned_LSOA', 'Total Volume']
df_long = df_long.merge(total_counts, on='Assigned_LSOA')
df_long['Color Intensity'] = df_long['Total Volume'] / df_long['Total Volume'].m
df_long.sort_values(by=['Color Intensity', 'Date', 'Assigned_LSOA'], ascending=[F

fig = px.line(df_long, x='Date', y='Call Counts', color='Assigned_LSOA', labels={'As
color_discrete_sequence=px.colors.sequential.algae_r)

fig.update_traces(mode='lines+markers', marker=dict(size=8), opacity=0.4)

fig.update_layout(plot_bgcolor='white',
    xaxis=dict(showgrid=True, gridcolor='lightgrey', tickformat='%b
    range=[df_long['Date'].min(), df_long['Date'].max()]),
```

```

        yaxis=dict(showgrid=True, gridcolor='lightgrey', showline=True,
                    range=[0, 36]),
        title=dict(text='Monthly Trends of GP interactions for mental
                        y= 0.95,x= 0.5,
                        font=dict(size=20, family='Arial Black')),
        showlegend=False)

```

```

fig.show()
fig.to_html('../../results/monthly_GP.html')

```

```

In [13]: df_police_calls = pd.read_csv('../../data/police_monthly_call_vol.csv', index_col=0)
df_police_calls = df_police_calls.iloc[:, 1:]
new_dates = [convert_multiindex_to_date(col) for col in df_police_calls.columns]
df_police_calls.columns = sorted(new_dates)

```

```

In [14]: df_police_calls.index.names = ['Assigned_LSOA']
df_policecalls_long = df_police_calls.reset_index().melt(id_vars='Assigned_LSOA')

```

```

In [15]: lsoa_police_gp = df_policecalls_long.merge(df_long, on=['Assigned_LSOA', 'Date'], how='left')
lsoa_police_gp['diff_gp_police_calls'] = lsoa_police_gp['Call Counts'] - lsoa_police_gp['Total Difference in Calls']

```

```

In [ ]: total_counts = lsoa_police_gp.groupby('Assigned_LSOA')['diff_gp_police_calls'].sum()
total_counts.columns = ['Assigned_LSOA', 'Total Difference in Calls']
lsoa_police_gp = lsoa_police_gp.merge(total_counts, on='Assigned_LSOA')
lsoa_police_gp['Diff_intensity'] = lsoa_police_gp['Total Difference in Calls'] / lsoa_police_gp['diff_gp_police_calls']
lsoa_police_gp.sort_values(by=['Diff_intensity', 'Date', 'Assigned_LSOA'], inplace=True)

```

```

fig = px.line(lsoa_police_gp, x='Date', y='diff_gp_police_calls', color='Assigned_LSOA',
              color_discrete_sequence=px.colors.sequential.OrRd_r)

```

```

fig.update_traces(mode='lines+markers', marker=dict(size=8), opacity=0.4)

```

```

# fig.add_trace(go.Scatter(
#     x=[lsoa_police_gp['Date'].min(), lsoa_police_gp['Date'].max()],
#     y=[0, 0],
#     mode='lines',
#     line=dict(width=12, color='black'),
#     name='Reference Line',
#     showlegend=True
# ))

```

```

fig.add_hline(y=0, line=dict(color='black', width=2, dash='dash'))

```

```

fig.update_layout(plot_bgcolor='white',
                  xaxis=dict(showgrid=True, gridcolor='lightgrey', tickformat='%b %Y',
                              range=[lsoa_police_gp['Date'].min(), lsoa_police_gp['Date'].max()]),
                  yaxis=dict(showgrid=True, gridcolor='lightgrey', showline=True,
                              range=[-30, 30]),
                  title=dict(text='Monthly Trends of difference between GP interactions and police calls
                                y= 0.95,x= 0.5,
                                font=dict(size=20, family='Arial Black')),
                  showlegend=False)

```

```

fig.show()
fig.to_html('../../results/monthly_diff_GP_Pol.html')

```

```

In [17]: # Define conditions and corresponding values to categorise data into the Financial Year
lsoa_police_gp['Date'] = pd.to_datetime(lsoa_police_gp['Date'])
conditions = [
    (lsoa_police_gp.Date.dt.year == 2017) & (lsoa_police_gp.Date.dt.month.between(1, 12)),
    (lsoa_police_gp.Date.dt.year == 2018) & (lsoa_police_gp.Date.dt.month.between(1, 12)),
    (lsoa_police_gp.Date.dt.year == 2019) & (lsoa_police_gp.Date.dt.month.between(1, 12)),
    (lsoa_police_gp.Date.dt.year == 2020) & (lsoa_police_gp.Date.dt.month.between(1, 12)),
    (lsoa_police_gp.Date.dt.year == 2021) & (lsoa_police_gp.Date.dt.month.between(1, 12)),
    (lsoa_police_gp.Date.dt.year == 2022) & (lsoa_police_gp.Date.dt.month.between(1, 12)),
    (lsoa_police_gp.Date.dt.year == 2023) & (lsoa_police_gp.Date.dt.month.between(1, 12)),
    (lsoa_police_gp.Date.dt.year == 2024) & (lsoa_police_gp.Date.dt.month.between(1, 12)),
    (lsoa_police_gp.Date.dt.year == 2025) & (lsoa_police_gp.Date.dt.month.between(1, 12))
]

```

```

(lsoa_police_gp.Date.dt.year == 2018) & (lsoa_police_gp.Date.dt.month.betwe
(lsoa_police_gp.Date.dt.year == 2019) & (lsoa_police_gp.Date.dt.month.betwe
(lsoa_police_gp.Date.dt.year == 2019) & (lsoa_police_gp.Date.dt.month.betwe
(lsoa_police_gp.Date.dt.year == 2020) & (lsoa_police_gp.Date.dt.month.betwe
(lsoa_police_gp.Date.dt.year == 2020) & (lsoa_police_gp.Date.dt.month.betwe
(lsoa_police_gp.Date.dt.year == 2021) & (lsoa_police_gp.Date.dt.month.betwe
(lsoa_police_gp.Date.dt.year == 2021) & (lsoa_police_gp.Date.dt.month.betwe
(lsoa_police_gp.Date.dt.year == 2022) & (lsoa_police_gp.Date.dt.month.betwe

values = ['17-18', '17-18', '18-19', '18-19', '19-20', '19-20', '20-21', '20-21'

lsoa_police_gp['YearTrunk'] = np.select(conditions, values, default='Other')

```

```

In [18]: yearly_lsoa_police_gp = lsoa_police_gp.groupby(['Assigned_LSOA', 'YearTrunk'])['d

yearly_lsoa_police_gp_wide = yearly_lsoa_police_gp.pivot(index='Assigned_LSOA',

# Compute 'Total' column summing across specific year columns
yearly_lsoa_police_gp_wide['Total'] = yearly_lsoa_police_gp_wide[['17-18', '18-1
summary_row = yearly_lsoa_police_gp_wide[['17-18', '18-19', '19-20', '20-21', '2

```

```

In [19]: yearly_lsoa_police_gp_wide = pd.concat([yearly_lsoa_police_gp_wide, pd.DataFrame
yearly_lsoa_police_gp_wide.rename(index={0: 'Column Total'}, inplace=True)

yearly_lsoa_police_gp_wide_sorted = yearly_lsoa_police_gp_wide.sort_values(by='T
yearly_lsoa_police_gp_wide_sorted.rename(columns={'index': 'LSOA'}, inplace=True)

yearly_lsoa_police_gp_wide_sorted = yearly_lsoa_police_gp_wide_sorted.merge(df_1

gdf_diff_volume = gpd.GeoDataFrame(yearly_lsoa_police_gp_wide_sorted, geometry=ye

```

```

In [20]: sns.set_theme()

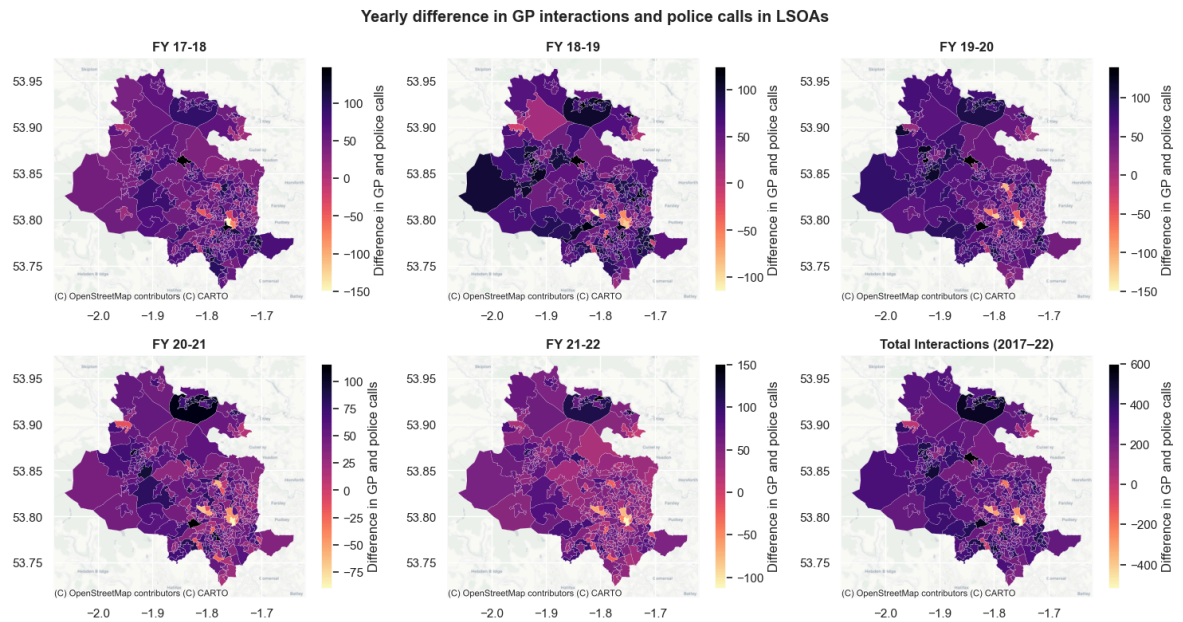
columns = ['17-18', '18-19', '19-20', '20-21', '21-22', 'Total']
titles = ['FY 17-18', 'FY 18-19', 'FY 19-20', 'FY 20-21', 'FY 21-22', 'Total Int

# Create choropleth maps for interaction counts
fig, axes = plt.subplots(2, 3, figsize=(15, 8))

for ax, column, title in zip(axes.flatten(), columns, titles):
    gdf_diff_volume.plot(
        column=column,
        cmap='magma_r',
        ax=ax,
        linewidth=0.1,
        legend=True,
        legend_kws={'label': "Difference in GP and police calls", 'orientation'
    )
    ax.set_title(title, fontweight='bold')
    cx.add_basemap(ax, crs=gdf_diff_volume.crs.to_string(), source=cx.providers.

# Show the plot
plt.suptitle('Yearly difference in GP interactions and police calls in LSOAs', fo
plt.tight_layout()
plt.show()

```

```
In [21]: df_police_calls = pd.read_csv('../data/BD_Policedata_2017_22_all.csv', index_c
df_police_calls['incident_start_date'] = pd.to_datetime(df_police_calls['incident
df_police_calls['incident_end_date'] = pd.to_datetime(df_police_calls['incident_e
```

```
In [22]: df_police_calls.info()
```

```
<class 'pandas.core.frame.DataFrame'>
```

```
Index: 24428 entries, 327 to 27302
```

```
Data columns (total 34 columns):
```

#	Column	Non-Null Count	Dtype
0	lsoa_name1	14130 non-null	object
1	Index	0 non-null	float64
2	lsoa1	14130 non-null	object
3	lsoa	24428 non-null	object
4	incident_ref	14130 non-null	object
5	reswodep	24428 non-null	bool
6	initial_grade	5326 non-null	object
7	attended	24428 non-null	bool
8	incident_sub_type	4280 non-null	object
9	disposition_1_code	14130 non-null	float64
10	qualifiers	24428 non-null	object
11	final_grade	24428 non-null	object
12	incident_end_date	24428 non-null	datetime64[ns, UTC]
13	cb_date_added	24428 non-null	object
14	incident_type	24428 non-null	object
15	lsoa_name	24428 non-null	object
16	incident_start_date	24428 non-null	datetime64[ns, UTC]
17	postcode_district	10298 non-null	object
18	incident_id	10298 non-null	float64
19	incident_duration	24428 non-null	float64
20	is_duplicate	24428 non-null	bool
21	incident_start_year	24428 non-null	int64
22	incident_start_month	24428 non-null	object
23	incident_start_day_num	24428 non-null	int64
24	incident_start_day	24428 non-null	object
25	incident_start_hour	24428 non-null	int64
26	incident_end_year	24428 non-null	int64
27	incident_end_month	24428 non-null	object
28	incident_end_day_num	24428 non-null	int64
29	incident_end_day	24428 non-null	object
30	incident_end_hour	24428 non-null	int64
31	Date_agg	24428 non-null	object
32	YearTrunk	24428 non-null	object
33	CrimeTag	24428 non-null	object

```
dtypes: bool(3), datetime64[ns, UTC](2), float64(4), int64(6), object(19)
```

```
memory usage: 6.0+ MB
```

```
In [23]: df_police_calls['start_date'] = df_police_calls['incident_start_date'].dt.date
df_police_calls_daily = df_police_calls.groupby('start_date').size().reset_index
```

```
In [24]: gp_lsoa_df_daily = gp_lsoa_df.groupby('DateEventRecorded').size().reset_index(na
```

```
In [25]: df_police_calls_daily['start_date'] = pd.to_datetime(df_police_calls_daily['start_
gp_lsoa_df_daily['DateEventRecorded'] = pd.to_datetime(gp_lsoa_df_daily['DateEve
```

```
In [26]: df_gp_police_all = df_police_calls_daily.merge(gp_lsoa_df_daily, left_on='start_d
```

```
In [27]: df_gp_police_all['diff_gp_police_calls'] = df_gp_police_all['GP_interaction_coun
```

```
In [28]: df_gp_police_calender = df_gp_police_all[['start_date', 'diff_gp_police_calls']]
df_gp_police_calender.set_index('start_date', inplace=True)
```

```
In [29]: df_gp_police_calender_v1 = df_gp_police_calender['diff_gp_police_calls'].rename(df_gp_police_calender_v1)
```

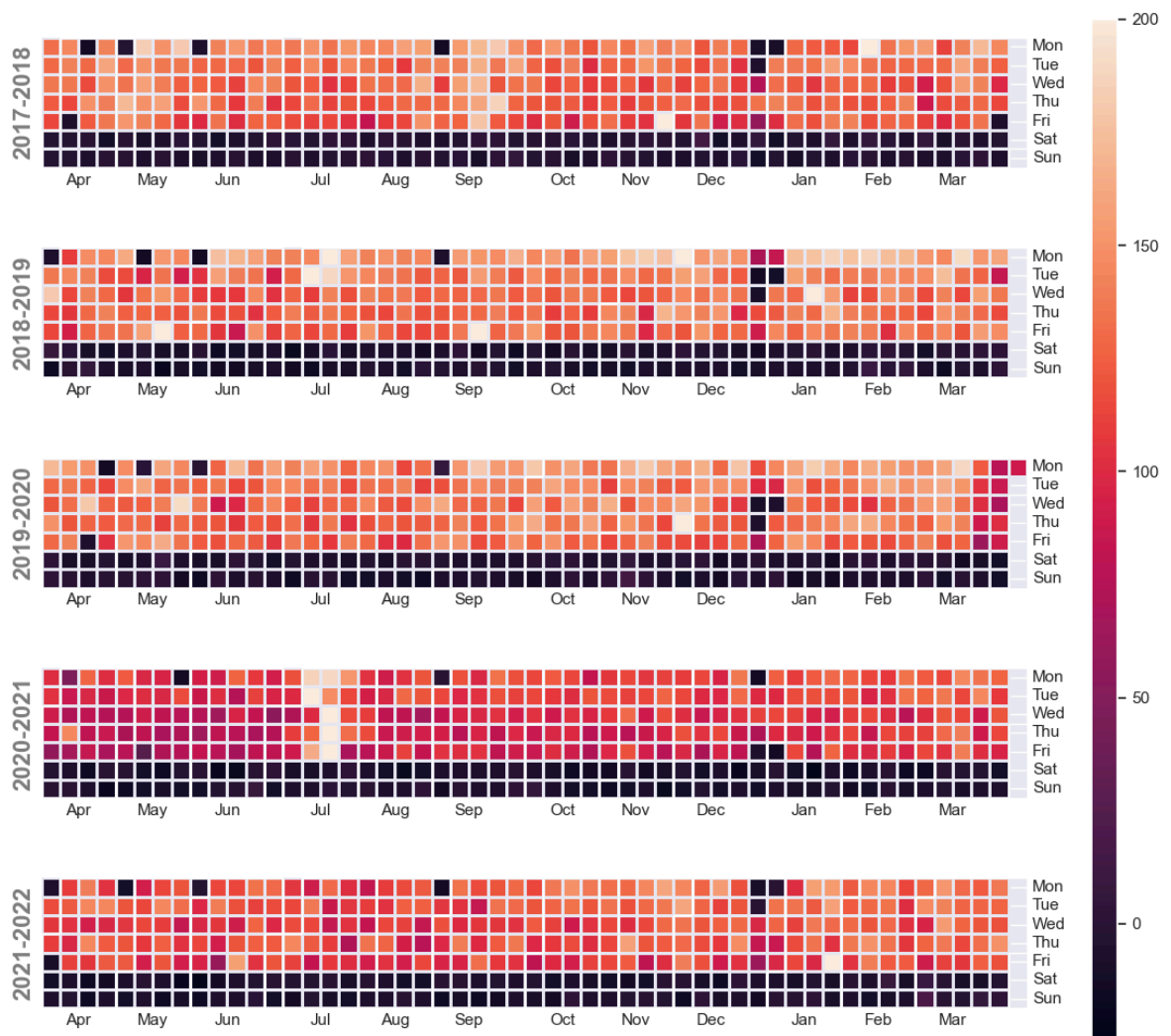
```
Out[29]: start_date
2017-04-01      -3.0
2017-04-02      -3.0
2017-04-03     132.0
2017-04-04     130.0
2017-04-05     137.0
...
2022-03-27       3.0
2022-03-28     138.0
2022-03-29     126.0
2022-03-30     118.0
2022-03-31     -14.0
Length: 1826, dtype: float64
```

```
In [30]: import sys
sys.path.append('../')
import importlib
from python import plot_calendar_heatmap
importlib.reload(plot_calendar_heatmap)

plot_calendar_heatmap.yearmonthdayplot(df_gp_police_calender_v1, edgecolor=None,
```

```
Out[30]: (<Figure size 1200x1000 with 6 Axes>,
array([<Axes: ylabel='2017-2018'>, <Axes: ylabel='2018-2019'>,
       <Axes: ylabel='2019-2020'>, <Axes: ylabel='2020-2021'>,
       <Axes: ylabel='2021-2022'>], dtype=object))
```

The difference in GP interactions and police calls in Bradford



Correlation between GP interactions and police calls for each LSOA

Monthwise correlation

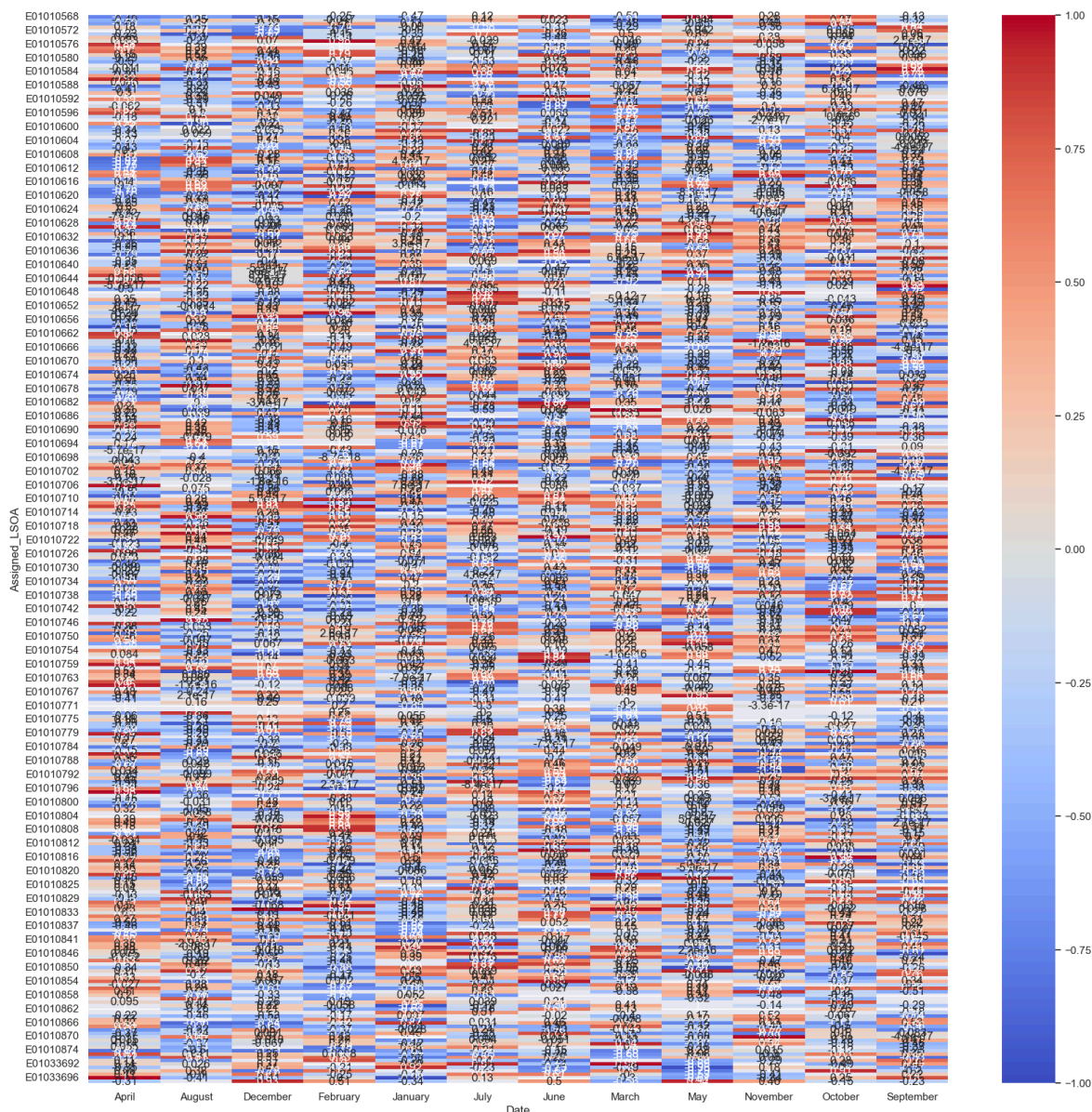
```
In [31]: from scipy.stats import pearsonr, spearmanr

correlation_df = lsoa_police_gp.groupby(['Assigned_LSOA', lsoa_police_gp['Date']].
    lambda group: group[['Police Call Counts', 'Call Counts']].corr().iloc[0,1]
).reset_index(name='pearson_correlation')

In [32]: heatmap_data = correlation_df.pivot(index='Assigned_LSOA', columns='Date', value=
    'pearson_correlation')

plt.figure(figsize=(20, 20))
sns.heatmap(heatmap_data, annot=True, cmap='coolwarm', center=0)

Out[32]: <Axes: xlabel='Date', ylabel='Assigned_LSOA'>
```



```
In [33]: correlation_df_lsoa = correlation_df.merge(df_lsoa_bradford[['LSOA11CD','geometr
```

```
In [34]: correlation_df_lsoa_pivot = correlation_df_lsoa.pivot(index='Assigned_LSOA', colu
correlation_df_lsoa_pivot = correlation_df_lsoa_pivot.merge(df_lsoa_bradford[['L
gdf_correlation_plot = gpd.GeoDataFrame(correlation_df_lsoa_pivot, geometry=corre
```

```
In [35]: sns.set_theme()

columns = ['January', 'February', 'March', 'April', 'May', 'June', 'July', 'August'
titles = ['Jan', 'Feb', 'Mar', 'Apr', 'May', 'Jun', 'Jul', 'Aug', 'Sep', 'Oct', 'Nov']

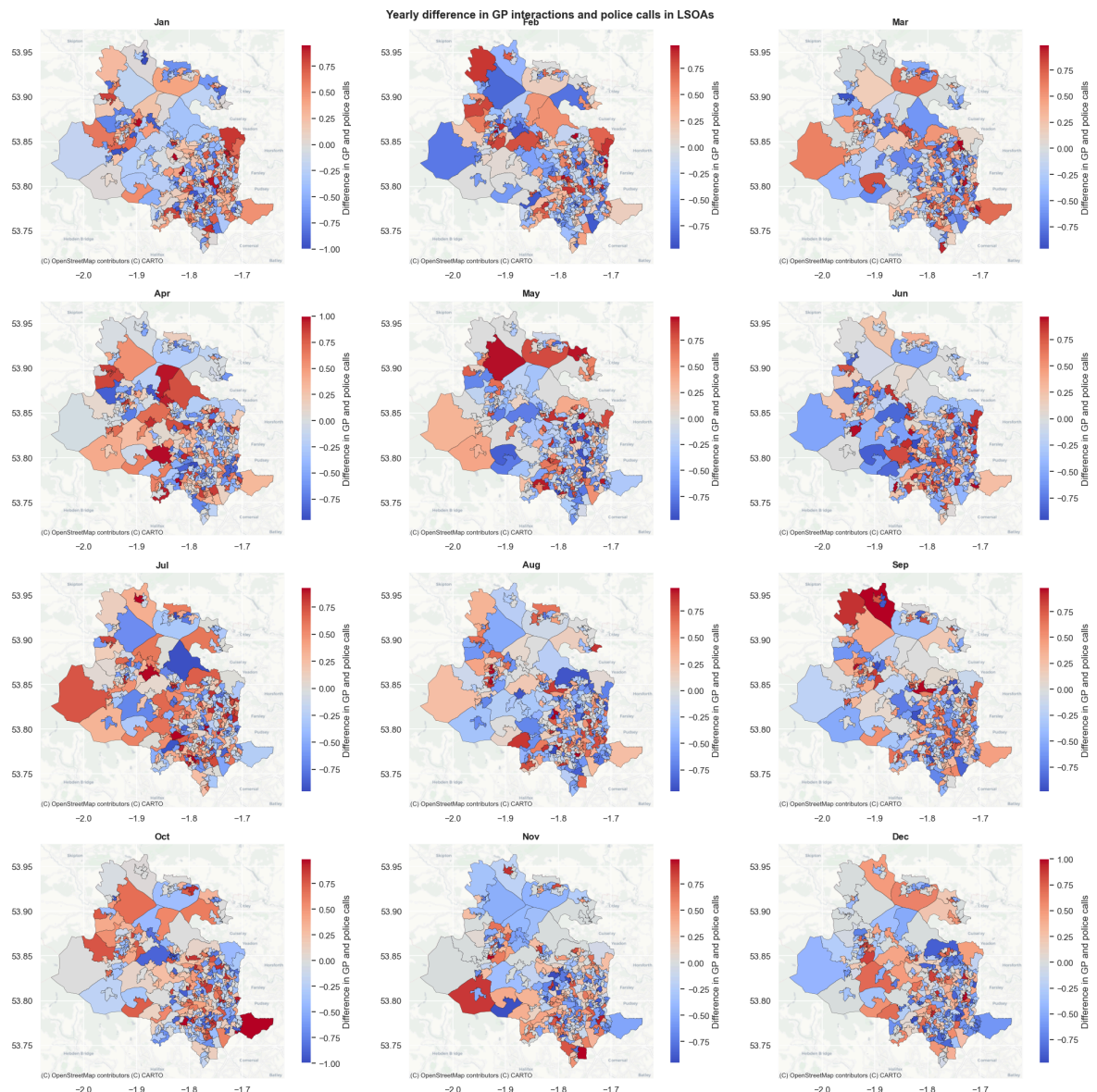
# Create choropleth maps for interaction counts
fig, axes = plt.subplots(4, 3, figsize=(20,20))

for ax, column, title in zip(axes.flatten(), columns, titles):
    gdf_correlation_plot.plot(
        column=column,
        cmap='coolwarm',
        ax=ax,
        linewidth=0.2,
        edgecolor='black',
        legend=True,
        legend_kws={'label': "Difference in GP and police calls", 'orientation'
```



```
)
ax.set_title(title,fontweight='bold')
cx.add_basemap(ax, crs=gdf_correlation_plot.crs.to_string(), source=cx.provi

# Show the plot
plt.suptitle('Yearly difference in GP interactions and police calls in LSOAs',fo
plt.tight_layout()
plt.show()
```



Yearwise correlation

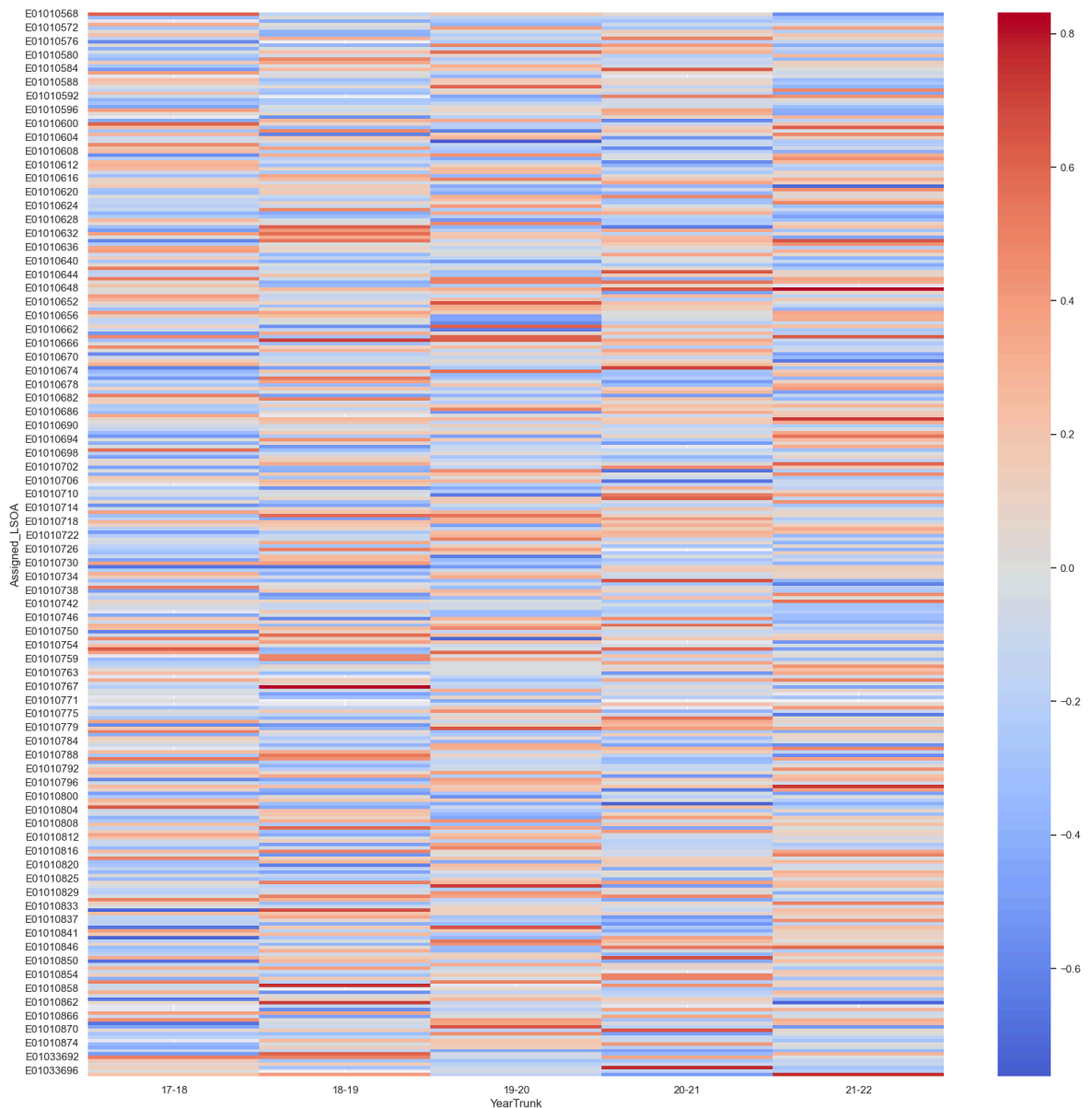
```
In [36]: # correlation_results_year = []
# # Group by LSOA and year
# grouped = lsoa_police_gp.groupby(['Assigned_LSOA', 'YearTrunk'])
# # Compute correlations for each group
# for (lsoa, year), group in grouped:
#     for p_var in police_counts:
#         for h_var in health_vars:
#             if group[p_var].count() > 1 and group[h_var].count() > 1:
#                 pearson_coef, _ = pearsonr(group[p_var], group[h_var])
#                 spearman_coef, _ = spearmanr(group[p_var], group[h_var])
#                 correlation_results_year.append({
```

```
#         'LSOA_code': lsoa,
#         'FY_year': year,
#         'police_variable': p_var,
#         'health_variable': h_var,
#         'pearson_correlation': pearson_coef,
#         'spearman_correlation': spearman_coef
#     })
correlation_df_year = lsoa_police_gp.groupby(['Assigned_LSOA', 'YearTrunk']).apply(
    lambda group: group[['Police Call Counts', 'Call Counts']].corr().iloc[0,1]
).reset_index(name='pearson_correlation')
```

```
In [37]: heatmap_data_year = correlation_df_year.pivot(index='Assigned_LSOA', columns='YearTrunk')

plt.figure(figsize=(20, 20))
sns.heatmap(heatmap_data_year, annot=False, cmap='coolwarm', center=0)
```

```
Out[37]: <Axes: xlabel='YearTrunk', ylabel='Assigned_LSOA'>
```



```
In [38]: # Correlation for each LSOA
correlation_df_overall = lsoa_police_gp.groupby('Assigned_LSOA').apply(
    lambda group: group[['Police Call Counts', 'Call Counts']].corr().iloc[0,1]
).reset_index(name='pearson_correlation')
```



```

In [39]: correlation_df_joined = pd.concat([correlation_df_year, correlation_df_overall], a

In [40]: correlation_df_joined['YearTrunk'] = correlation_df_joined['YearTrunk'].fillna('

In [41]: correlation_df_joined_pivot = correlation_df_joined.pivot(index='Assigned_LSOA',
correlation_df_lsoa_overall = correlation_df_joined_pivot.merge(df_lsoa_bradford
gdf_correlation_lsoa_year = gpd.GeoDataFrame(correlation_df_lsoa_overall, geometr

In [42]: sns.set_theme()

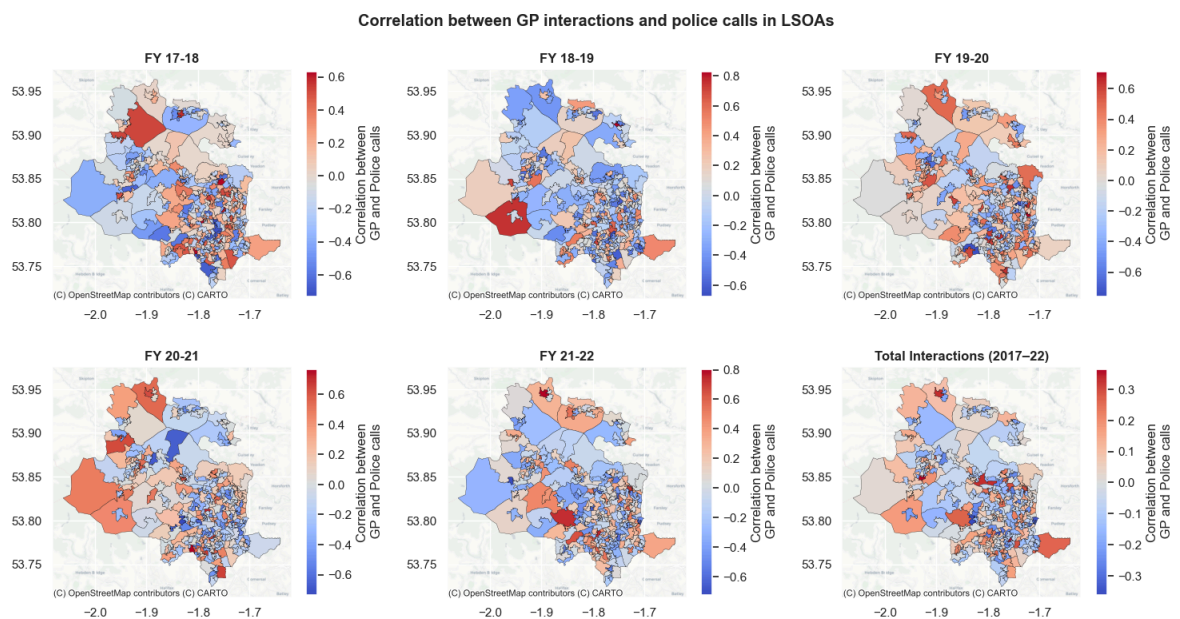
columns = ['17-18', '18-19', '19-20', '20-21', '21-22', 'Total']
titles = ['FY 17-18', 'FY 18-19', 'FY 19-20', 'FY 20-21', 'FY 21-22', 'Total Int

# Create choropleth maps for interaction counts
fig, axes = plt.subplots(2, 3, figsize=(15,8))

for ax, column, title in zip(axes.flatten(), columns, titles):
    gdf_correlation_lsoa_year.plot(
        column=column,
        cmap='coolwarm',
        ax=ax,
        linewidth=0.2,
        edgecolor='black',
        legend=True,
        legend_kws={'label': "Correlation between\n GP and Police calls", 'orie
    )
    ax.set_title(title, fontweight='bold')
    cx.add_basemap(ax, crs=gdf_correlation_lsoa_year.crs.to_string(), source=cx.

# Show the plot
plt.suptitle('Correlation between GP interactions and police calls in LSOAs', fon
plt.tight_layout()
plt.show()

```



```

In [43]: police_deciles = lsoa_police_gp.groupby('Assigned_LSOA')['Police Call Counts'].q
police_deciles.columns = [f"Police{int(q*100)}th percentile" for q in police_dec

```

```
In [44]: gp_mh_deciles = lsoa_police_gp.groupby('Assigned_LSOA')['Call Counts'].quantile(
gp_mh_deciles.columns = [f"GP{int(q*100)}th percentile" for q in gp_mh_deciles.c
```

```
In [45]: from scipy.spatial.distance import euclidean, cosine

decile_gp_police_df = pd.concat([gp_mh_deciles,police_deciles],axis=1)

decile_results = []
for lsoa, row in decile_gp_police_df.iterrows():
    area = lsoa
    GP_deciles = row[[f'GP{i}th percentile' for i in range(10, 100, 10)]]
    Police_deciles = row[[f'Police{i}th percentile' for i in range(10, 100, 10)]]
    euclid_dist = euclidean(GP_deciles, Police_deciles)
    cosine_sim = 1 - cosine(GP_deciles, Police_deciles)
    decile_results.append({'LSOA': area, 'euclidean_distance': euclid_dist, 'cos

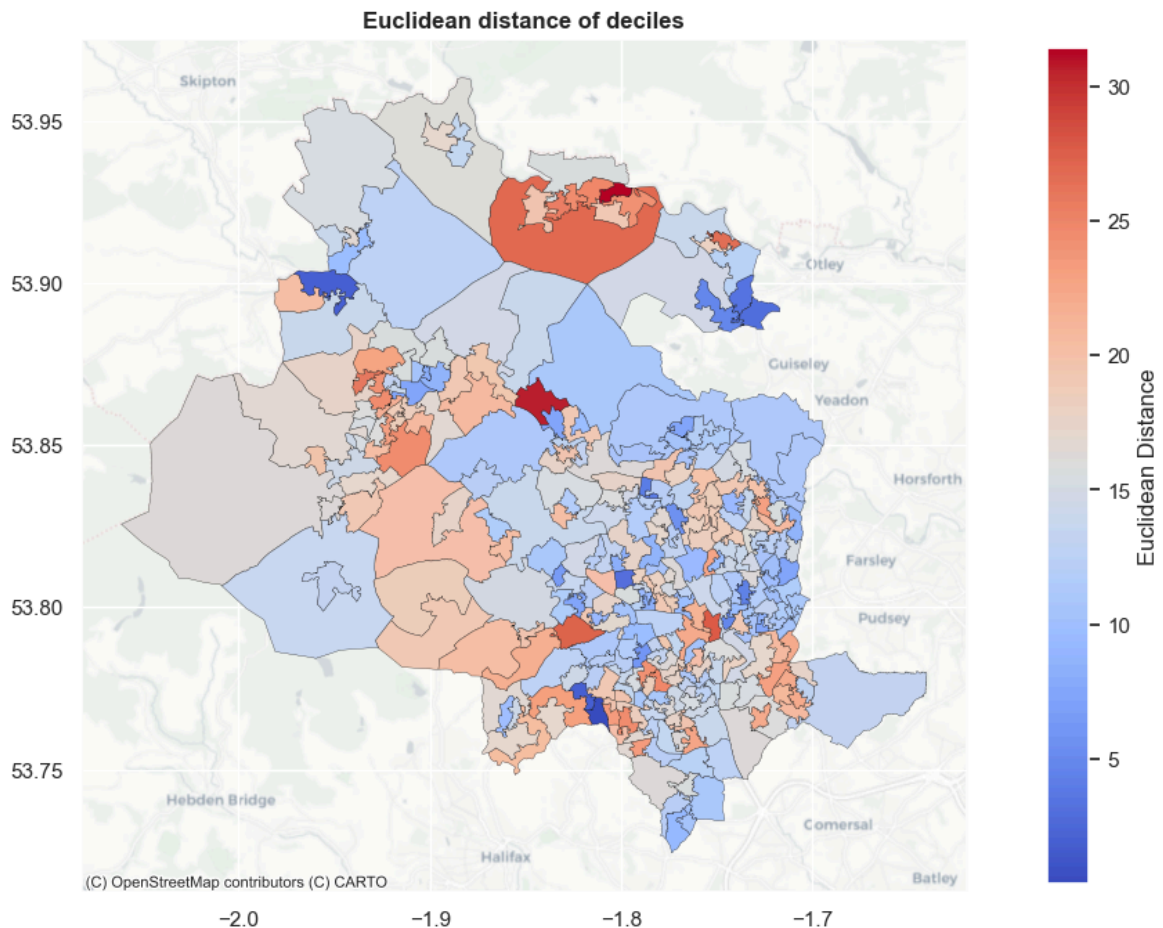
decile_results_df = pd.DataFrame(decile_results)
```

```
In [46]: similarity_df_lsoa_overall = decile_results_df.merge(df_lsoa_bradford[['LSOA11CD
gdf_similarity_lsoa = gpd.GeoDataFrame(similarity_df_lsoa_overall,geometry=simil
```

```
In [47]: fig, ax = plt.subplots(figsize=(15,8))

gdf_similarity_lsoa.plot(
    column='euclidean_distance',
    cmap='coolwarm',
    ax=ax,
    linewidth=0.2,
    edgecolor='black',
    legend=True,
    legend_kwds={'label': "Euclidean Distance", 'orientation': "vertical", '
)

ax.set_title('Euclidean distance of deciles',fontweight='bold')
cx.add_basemap(ax, crs=gdf_correlation_lsoa_year.crs.to_string(), source=cx.prov
```



```
In [48]: lsoa_overall = lsoa_police_gp.groupby('Assigned_LSOA').agg({'Police Call Counts':
lsoa_overall['diff'] = lsoa_overall['Police Call Counts']-lsoa_overall['Call Cou
lsoa_overall['Police_decile'] = pd.qcut(lsoa_overall['Police Call Counts'],10,la
lsoa_overall['GP_decile'] = pd.qcut(lsoa_overall['Call Counts'],10,labels=False)
lsoa_overall['diff_decile'] = lsoa_overall['Police_decile'] - lsoa_overall['GP_d

df_lsoa_overall_decile = lsoa_overall.merge(df_lsoa_bradford[['LSOA11CD','geomet
gdf_decile_lsoa = gpd.GeoDataFrame(df_lsoa_overall_decile,geometry=df_lsoa_ove
```

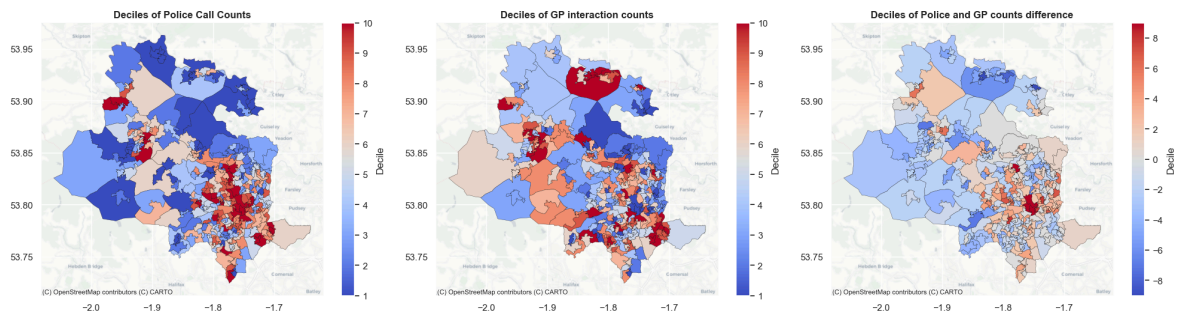
```
In [49]: sns.set_theme()

columns = ['Police_decile','GP_decile','diff_decile']
titles = ['Deciles of Police Call Counts', 'Deciles of GP interaction counts','D

# Create choropleth maps for deciles
fig, axes = plt.subplots(1, 3, figsize=(20,12))

for ax, column, title in zip(axes.flatten(), columns, titles):
    gdf_decile_lsoa.plot(
        column=column,
        cmap='coolwarm',
        ax=ax,
        linewidth=0.2,
        edgecolor='black',
        legend=True,
        legend_kws={'label': "Decile", 'orientation': "vertical", 'shrink': 0.3
    )
    ax.set_title(title,fontweight='bold')
    cx.add_basemap(ax, crs=gdf_correlation_lsoa_year.crs.to_string(), source=cx.
```

```
plt.tight_layout()
plt.show()
```



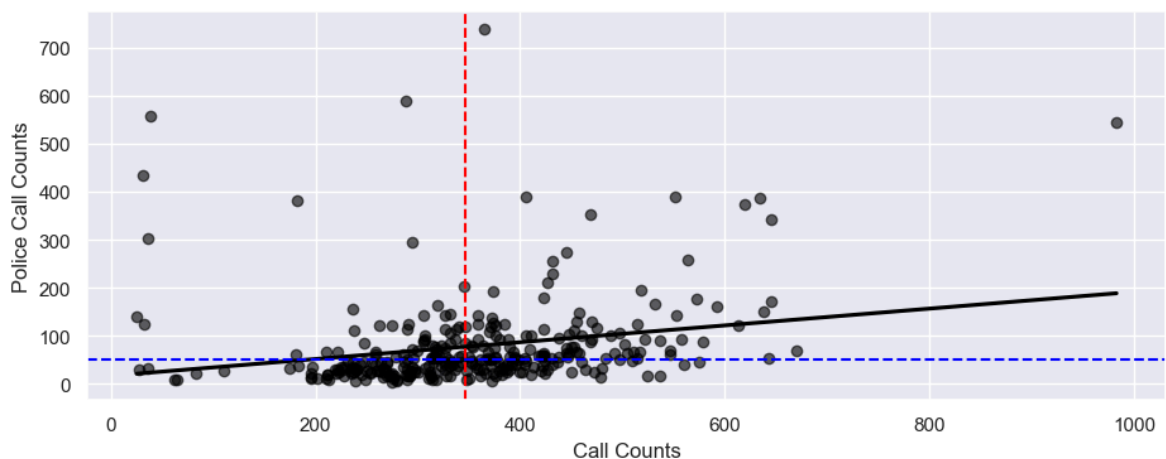
```
In [50]: sns.set_theme()

y_mean = lsoa_overall['Police Call Counts'].median()
x_mean = lsoa_overall['Call Counts'].median()

sns.regplot(
    x='Call Counts', y='Police Call Counts', data=lsoa_overall, ci=None, color='b
)

plt.axvline(x=x_mean, color='red', linestyle='--', label='Mean Police Call Count
plt.axhline(y=y_mean, color='blue', linestyle='--', label='Mean Call Counts')
```

Out[50]: <matplotlib.lines.Line2D at 0x12a73ba2350>



```
In [51]: def classify_quadrant(row):
    if row['Call Counts'] < x_mean and row['Police Call Counts'] >= y_mean:
        return 'Q1: Low GP, High Police Calls'
    elif row['Call Counts'] >= x_mean and row['Police Call Counts'] >= y_mean:
        return 'Q2: High GP, High Police Calls'
    elif row['Call Counts'] < x_mean and row['Police Call Counts'] < y_mean:
        return 'Q4: Low GP, Low Police Calls'
    else:
        return 'Q3: High GP, Low Police Calls'

lsoa_overall['Quadrant'] = lsoa_overall.apply(classify_quadrant, axis=1)
df_lsoa_overall_decile = lsoa_overall.merge(df_lsoa_bradford[['LSOA11CD'], 'geomet
gdf_decile_lsoa = gpd.GeoDataFrame(df_lsoa_overall_decile, geometry=df_lsoa_over

fig, ax = plt.subplots(figsize=(20,12))

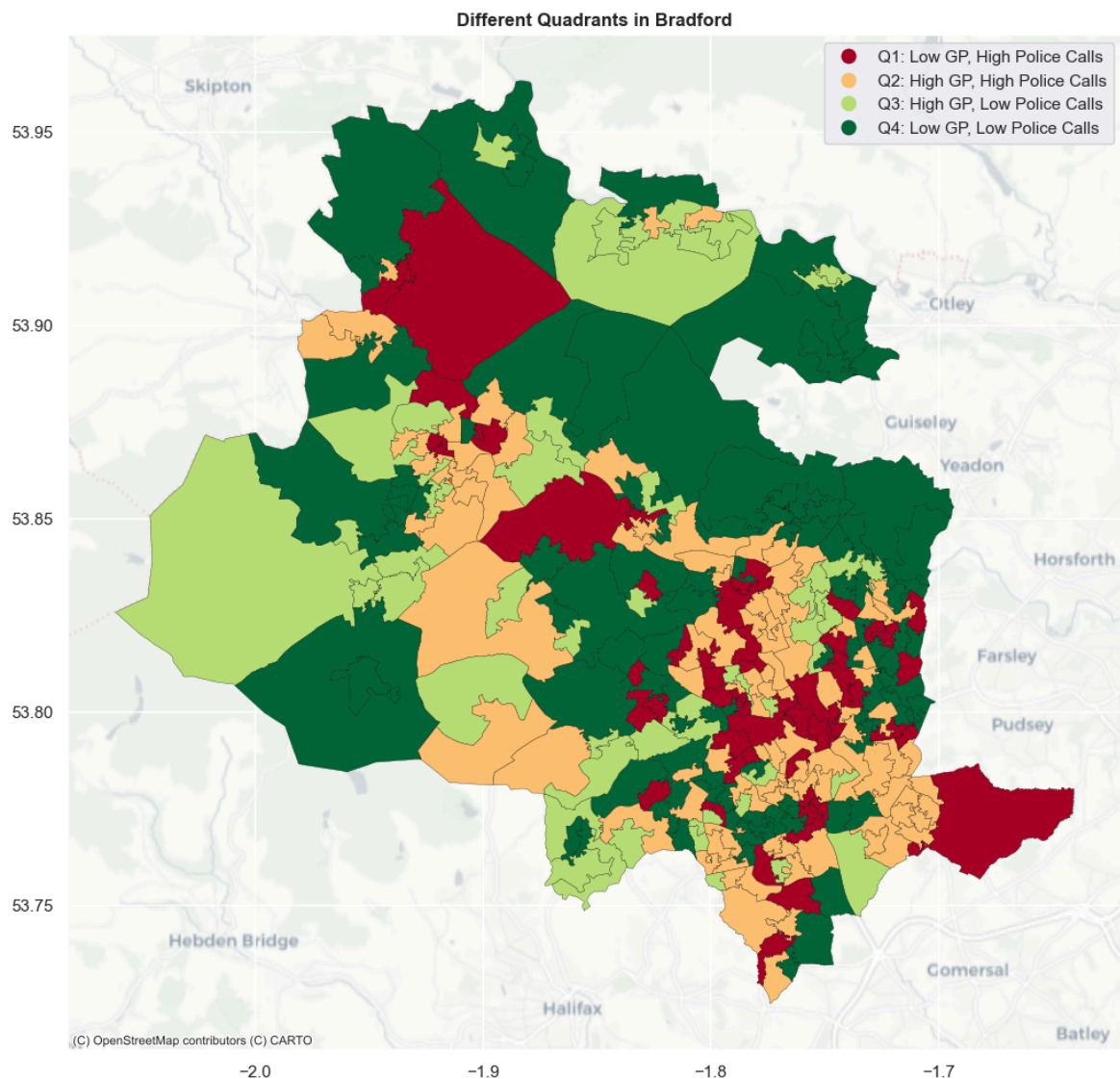
gdf_decile_lsoa.plot(
    column='Quadrant',
```

```

        cmap='RdYlGn',
        ax=ax,
        linewidth=0.2,
        edgecolor='black',
        legend=True
    )

    ax.set_title('Different Quadrants in Bradford', fontweight='bold')
    cx.add_basemap(ax, crs=gdf_decile_lsoa.crs.to_string(), source=cx.providers.Cart

```



```

In [52]: from scipy.stats import zscore

lsoa_overall['z_PoliceCall'] = zscore(lsoa_overall['Police Call Counts'])
lsoa_overall['z_CallCounts'] = zscore(lsoa_overall['Call Counts'])
lsoa_overall['z_diff'] = zscore(lsoa_overall['diff'])

df_lsoa_overall_zscore = lsoa_overall.merge(df_lsoa_bradford[['LSOA11CD', 'geomet
gdf_zscore_lsoa = gpd.GeoDataFrame(df_lsoa_overall_zscore, geometry=df_lsoa_overall

fig, ax = plt.subplots(figsize=(20,12))

gdf_zscore_lsoa.plot(
    column='z_diff',
    cmap='RdYlBu_r',
    ax=ax,
    vmin=-2,

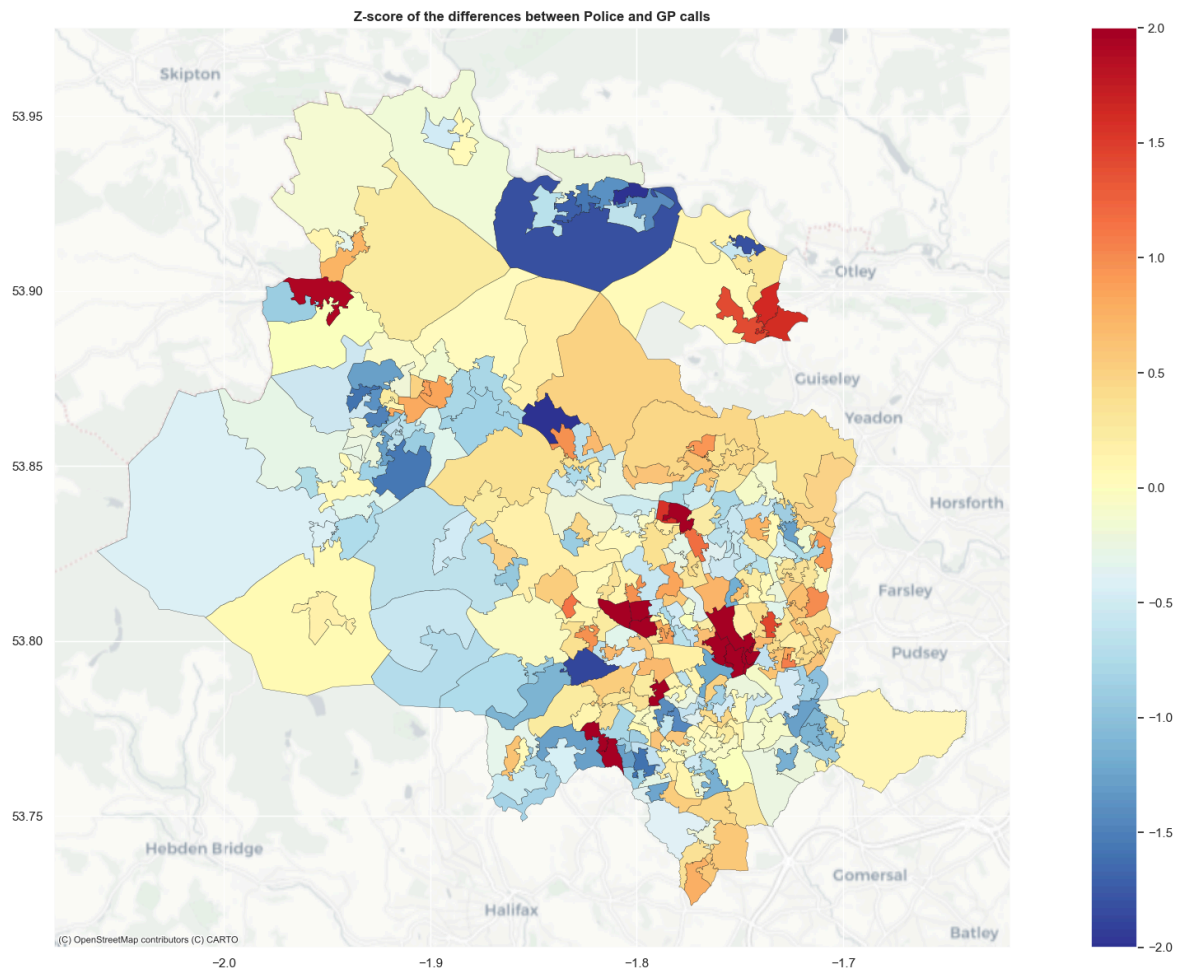
```



```

vmax=2,
linewidth=0.2,
edgecolor='black',
legend=True,
# legend_kws={'label': "zscore", 'orientation': "vertical", 'shrink': 0.9}
)
ax.set_title('Z-score of the differences between Police and GP calls',fontweight
cx.add_basemap(ax, crs=gdf_correlation_lsoa_year.crs.to_string(), source=cx.prov
plt.tight_layout()
plt.show()

```



```

In [53]: import statsmodels.api as sm

X = lsoa_overall['Call Counts']
X = sm.add_constant(X)
y = lsoa_overall['Police Call Counts']

model = sm.OLS(y, X).fit()

lsoa_overall['olsresid'] = model.resid
df_lsoa_overall_decile = lsoa_overall.merge(df_lsoa_bradford[['LSOA11CD', 'geomet
gdf_residual_lsoa = gpd.GeoDataFrame(df_lsoa_overall_decile, geometry=df_lsoa_ove

fig, ax = plt.subplots(figsize=(20,12))

gdf_residual_lsoa.plot(
    column='olsresid',
    cmap='RdYlBu_r',
    ax=ax,
    linewidth=0.2,

```

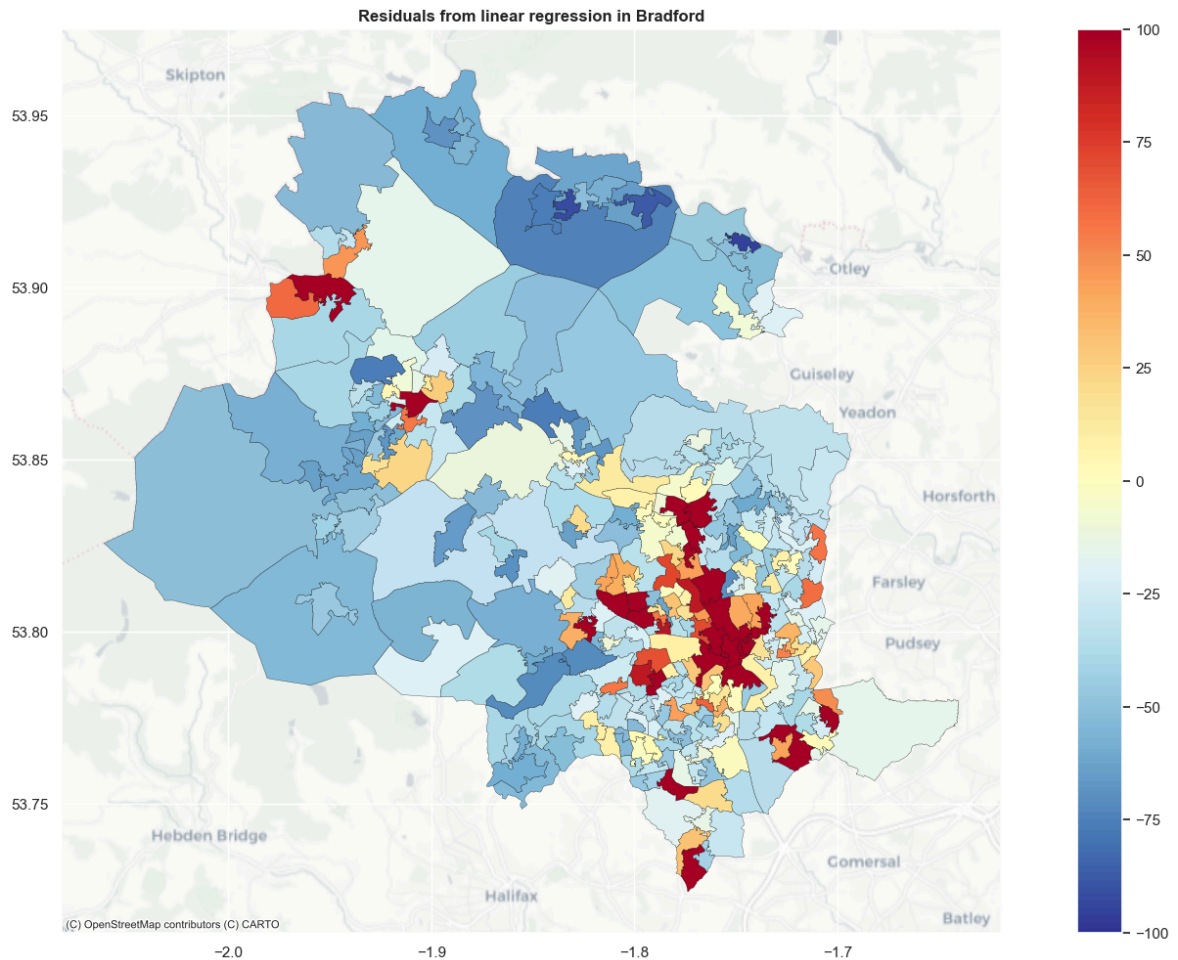


```

        edgecolor='black',
        legend=True,
        vmin=-100,
        vmax=100
    )

    ax.set_title('Residuals from linear regression in Bradford', fontweight='bold')
    cx.add_basemap(ax, crs=gdf_residual_lsoa.crs.to_string(), source=cx.providers.Ca

```



```

In [54]: X1 = lsoa_overall['z_CallCounts']
X1 = sm.add_constant(X1)
y1 = lsoa_overall['z_PoliceCall']

model2 = sm.OLS(y1, X1).fit()

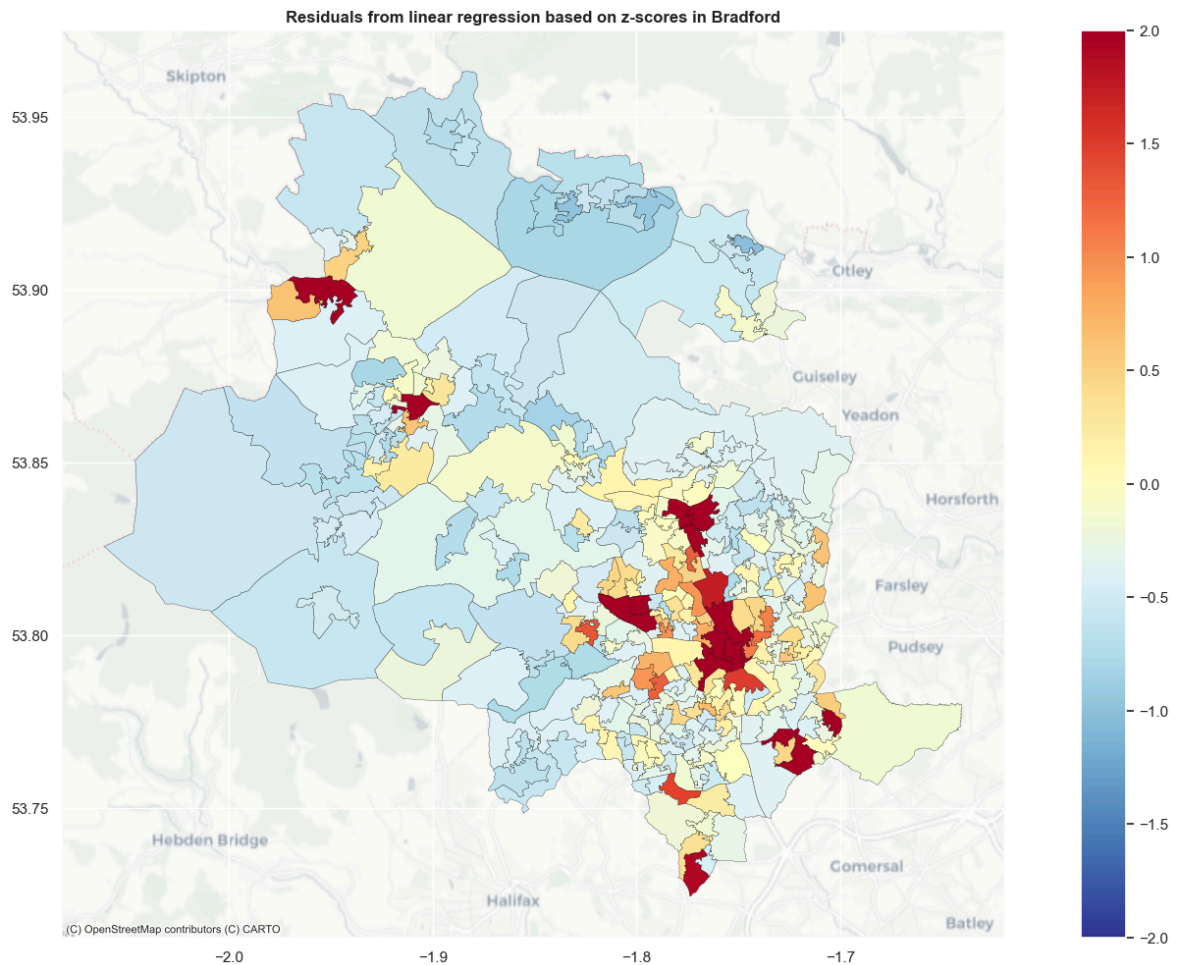
lsoa_overall['olsresid2'] = model2.resid
df_lsoa_overall_zscore_2 = lsoa_overall.merge(df_lsoa_bradford[['LSOA11CD', 'geom
gdf_lsoa_residual_zscore_2 = gpd.GeoDataFrame(df_lsoa_overall_zscore_2, geometry=

fig, ax = plt.subplots(figsize=(20,12))

gdf_lsoa_residual_zscore_2.plot(
    column='olsresid2',
    cmap='RdYlBu_r',
    ax=ax,
    linewidth=0.2,
    edgecolor='black',
    legend=True,
    vmin=-2,
    vmax=2
)

```

```
ax.set_title('Residuals from linear regression based on z-scores in Bradford', fo
cx.add_basemap(ax, crs=gdf_lsoa_residual_zscore_2.crs.to_string(), source=cx.pro
```



A **positive residual** in the model indicates that the **observed Police Call counts** is **higher than the predicted value** from the model. A **positive residual** means that the **actual number of police calls** in a given LSOA is **higher than what the model predicted** based on general call activity. This could suggest that the area has **more police-related incidents than expected**, given its overall call volume.

A **negative residual** indicates that the **actual number of police calls** in a given LSOA is **less than what the model predicted** based on overall call activity.

```
In [55]: lsoa_yearwise = lsoa_police_gp.groupby(['Assigned_LSOA', 'YearTrunk'])[['Call Cou

X = lsoa_yearwise['Call Counts']
X = sm.add_constant(X)
y = lsoa_yearwise['Police Call Counts']

model_3 = sm.OLS(y, X)
result_2 = model_3.fit()
```

```
In [56]: lsoa_yearwise['olsresid'] = result_2.resid
lsoa_yearwise_pivot = lsoa_yearwise.pivot(index='Assigned_LSOA', columns='YearTru
lsoa_yearwise_overall = lsoa_overall.merge(lsoa_yearwise_pivot, on='Assigned_LSO

df_lsoa_yearwise_overall = lsoa_yearwise_overall.merge(df_lsoa_bradford[['LSOA11
gdf_residual_yearwise = gpd.GeoDataFrame(df_lsoa_yearwise_overall, geometry=df_ls
```

```

sns.set_theme()

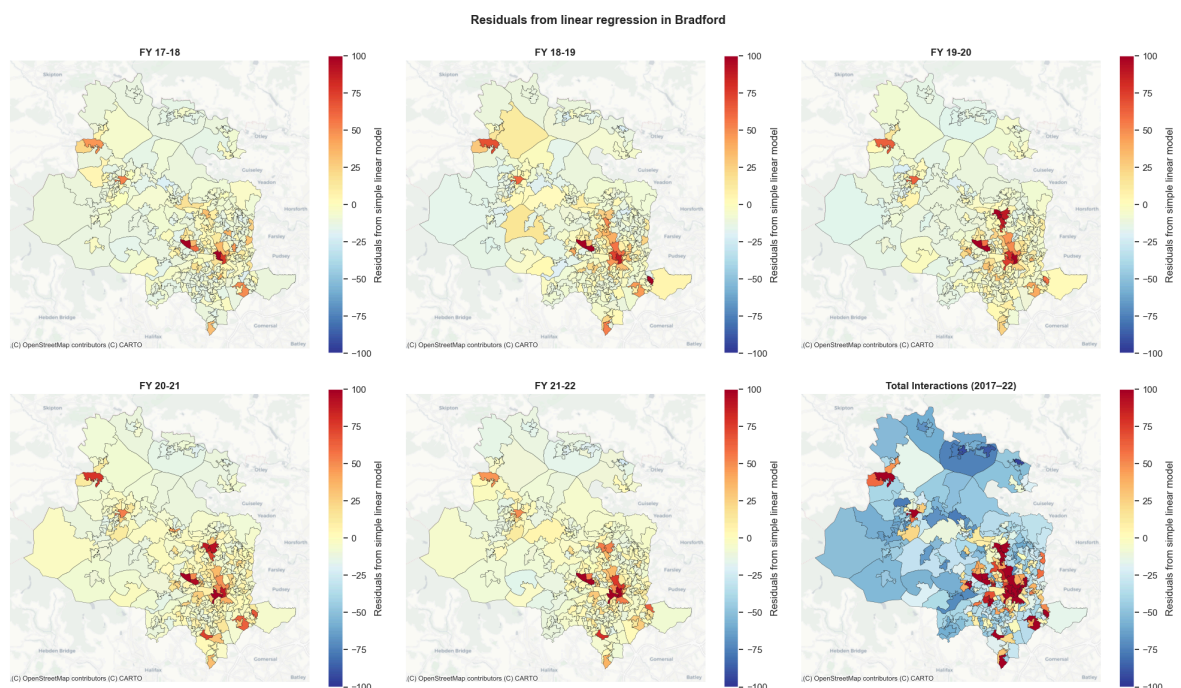
columns = ['17-18', '18-19', '19-20', '20-21', '21-22', 'olsresid']
titles = ['FY 17-18', 'FY 18-19', 'FY 19-20', 'FY 20-21', 'FY 21-22', 'Total Int

# Create choropleth maps for interaction counts
fig, axes = plt.subplots(2, 3, figsize=(20,12))

for ax, column, title in zip(axes.flatten(), columns, titles):
    gdf_residual_yearwise.plot(
        column=column,
        cmap='RdYlBu_r',
        ax=ax,
        linewidth=0.2,
        edgecolor='black',
        legend=True,
        legend_kwds={'label': "Residuals from simple linear model", 'orientation
vmin=-100,
vmax=100
    )
    ax.set_title(title, fontweight='bold')
    ax.set_axis_off()
    cx.add_basemap(ax, crs=gdf_residual_yearwise.crs.to_string(), source=cx.prov

# Show the plot
plt.suptitle('Residuals from linear regression in Bradford', fontweight='bold')
fig.tight_layout()
plt.show()

```



```

In [57]: from pysal.lib import weights
from pysal.explore import esda
from spplot import esda as esdaplot

w = weights.Queen.from_dataframe(gdf_residual_yearwise)
w.transform = "R"

lisa_pc = esda.moran.Moran_Local(gdf_residual_yearwise["Police Call Counts"], w)

```

```

gdf_residual_yearwise_mdcrs = gdf_residual_yearwise.to_crs(epsg=3857)

f, axs = plt.subplots(nrows=2, ncols=2, figsize=(20, 20))
axs = axs.flatten()

# Choropleth of Local statistics
ax = axs[0]
gdf_residual_yearwise_mdcrs.assign(
    Is=lisa_pc.Is
    # Plot choropleth of Local statistics
).plot(
    column="Is",
    cmap="plasma",
    scheme="quantiles",
    k=5,
    edgecolor="white",
    linewidth=0.1,
    alpha=0.75,
    legend=True,
    ax=ax,
)

# Quadrant categories
ax = axs[1]
esdaplot.lisa_cluster(lisa_pc, gdf_residual_yearwise_mdcrs, p=1, ax=ax)

# Significance map
ax = axs[2]
labels = pd.Series(
    1 * (lisa_pc.p_sim < 0.05), # Assign 1 if significant, 0 otherwise
    index=gdf_residual_yearwise_mdcrs.index
).map({1: "Significant", 0: "Non-Significant"})

gdf_residual_yearwise_mdcrs.assign(
    cl=labels
    # Plot choropleth of (non-)significant areas
).plot(
    column="cl",
    categorical=True,
    k=2,
    cmap="Paired",
    linewidth=0.1,
    edgecolor="white",
    legend=True,
    ax=ax,
)

# Cluster map
ax = axs[3]
# use a 5% significance level to select polygons as part of statistically signif
esdaplot.lisa_cluster(lisa_pc, gdf_residual_yearwise_mdcrs, p=0.05, ax=ax)

# Set title to each subplot
for i, ax in enumerate(axs.flatten()):
    ax.set_axis_off()
    ax.set_title(
        [
            "Local Statistics",
            "Local Moran Clusters",
            "Statistical Significance",

```

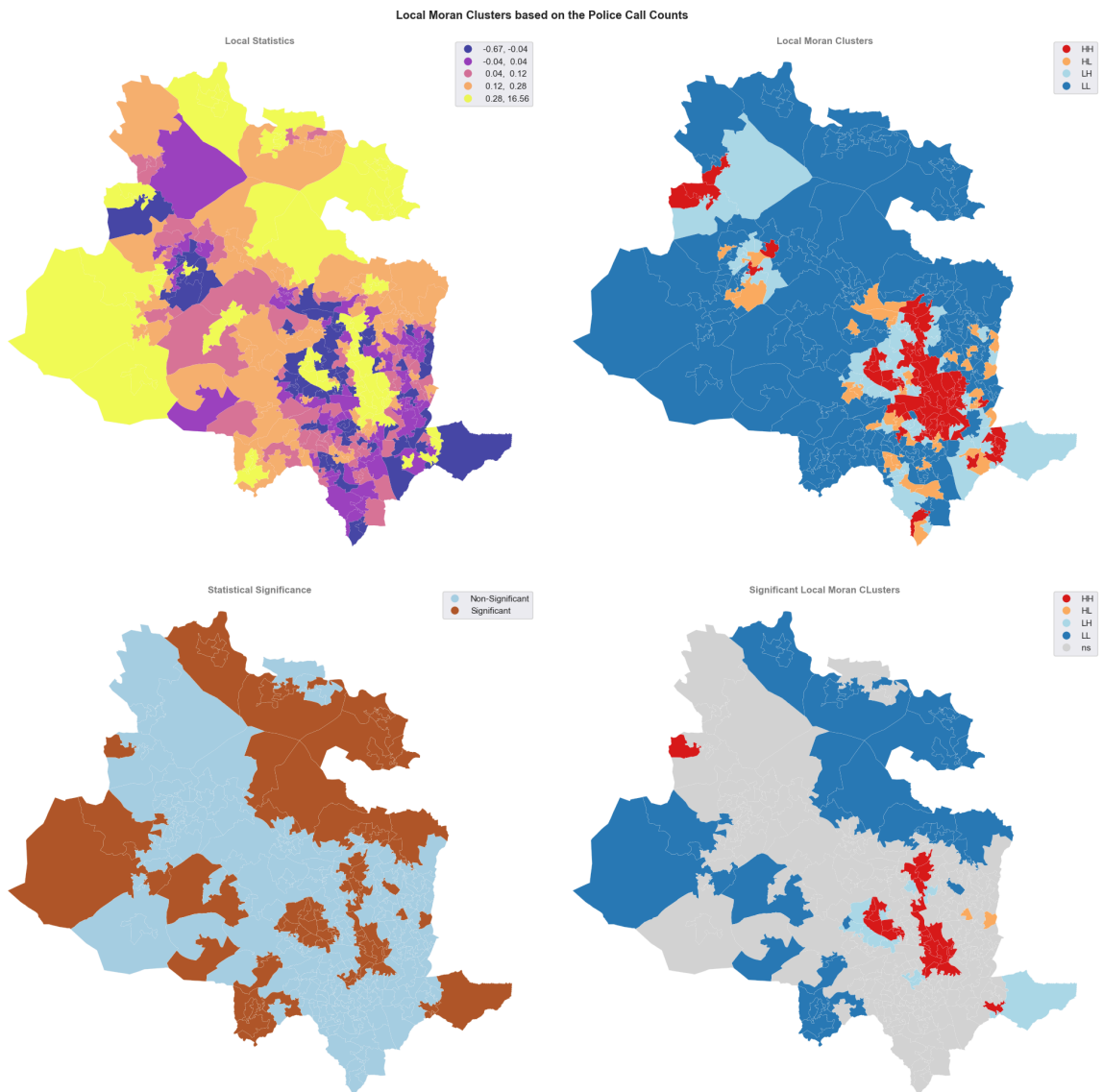
```

        "Significant Local Moran Clusters",
        ][i],
        y=0.98,
        color='gray',
        fontweight='bold'
    )

plt.suptitle('Local Moran Clusters based on the Police Call Counts', fontweight='bold')
plt.tight_layout()

plt.show()

```



```

In [58]: w = weights.Queen.from_dataframe(gdf_residual_yearwise)
w.transform = "R"

lisa_gpc = esda.moran.Moran_Local(gdf_residual_yearwise["Call Counts"], w)

gdf_residual_yearwise_mdcrs = gdf_residual_yearwise.to_crs(epsg=3857)

f, axs = plt.subplots(nrows=2, ncols=2, figsize=(20, 20))
axs = axs.flatten()

# Choropleth of local statistics
ax = axs[0]
gdf_residual_yearwise_mdcrs.assign(

```

```

    Is=lisa_gpc.Is
    # Plot choropleth of local statistics
).plot(
    column="Is",
    cmap="plasma",
    scheme="quantiles",
    k=5,
    edgecolor="white",
    linewidth=0.1,
    alpha=0.75,
    legend=True,
    ax=ax,
)

# Quadrant categories
ax = axs[1]
esdaplot.lisa_cluster(lisa_gpc, gdf_residual_yearwise_mdcrs, p=1, ax=ax)

# Significance map
ax = axs[2]
labels = pd.Series(
    1 * (lisa_gpc.p_sim < 0.05), # Assign 1 if significant, 0 otherwise
    index=gdf_residual_yearwise_mdcrs.index
).map({1: "Significant", 0: "Non-Significant"})

gdf_residual_yearwise_mdcrs.assign(
    cl=labels
    # Plot choropleth of (non-)significant areas
).plot(
    column="cl",
    categorical=True,
    k=2,
    cmap="Paired",
    linewidth=0.1,
    edgecolor="white",
    legend=True,
    ax=ax,
)

# Cluster map
ax = axs[3]
# use a 5% significance level to select polygons as part of statistically signif
esdaplot.lisa_cluster(lisa_gpc, gdf_residual_yearwise_mdcrs, p=0.05, ax=ax)

# Set title to each subplot
for i, ax in enumerate(axs.flatten()):
    ax.set_axis_off()
    ax.set_title(
        [
            "Local Statistics",
            "Local Moran Clusters",
            "Statistical Significance",
            "Significant Local Moran Clusters",
        ][i],
        y=0.98,
        color='gray',
        fontweight='bold'
    )

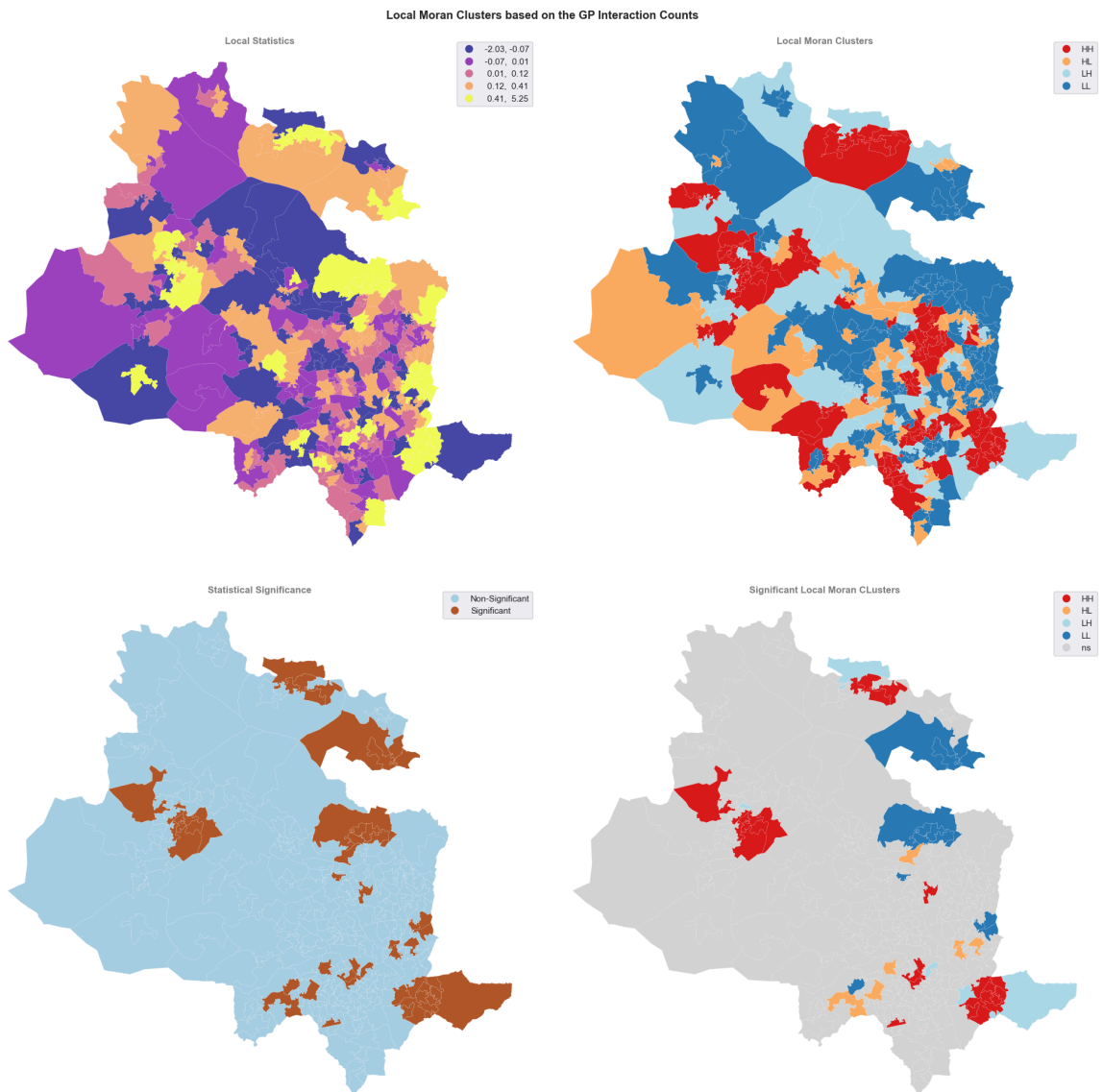
plt.suptitle('Local Moran Clusters based on the GP Interaction Counts', fontweigh

```



```
f.tight_layout()
```

```
plt.show()
```



```
In [59]: w = weights.Queen.from_dataframe(gdf_residual_yearwise)
w.transform = "R"

gdf_residual_yearwise["Police_calls_per_100_GP"] = (gdf_residual_yearwise["Police_calls_per_100_GP"] - gdf_residual_yearwise["Police_calls_per_100_GP"].mean()) / gdf_residual_yearwise["Police_calls_per_100_GP"].std()

lisa_gpc_est = esda.moran.Moran_Local(gdf_residual_yearwise["Police_calls_per_100_GP"], w)

gdf_residual_yearwise_mdcrs = gdf_residual_yearwise.to_crs(epsg=3857)

f, axs = plt.subplots(nrows=2, ncols=2, figsize=(20, 20))
axs = axs.flatten()

# Choropleth of Local statistics
ax = axs[0]
gdf_residual_yearwise_mdcrs.assign(
    Is=lisa_gpc_est.Is
).plot(
    column="Is",
    cmap="plasma",
    scheme="quantiles",
```

```

        k=5,
        edgecolor="white",
        linewidth=0.1,
        alpha=0.75,
        legend=True,
        ax=ax,
    )

    # Quadrant categories
    ax = axs[1]
    esdplot.lisa_cluster(lisa_gpc_est, gdf_residual_yearwise_mdcrs, p=1, ax=ax)

    # Significance map
    ax = axs[2]
    labels = pd.Series(
        1 * (lisa_gpc_est.p_sim < 0.05), # Assign 1 if significant, 0 otherwise
        index=gdf_residual_yearwise_mdcrs.index
    ).map({1: "Significant", 0: "Non-Significant"})

    gdf_residual_yearwise_mdcrs.assign(
        cl=labels
        # Plot choropleth of (non-)significant areas
    ).plot(
        column="cl",
        categorical=True,
        k=2,
        cmap="Paired",
        linewidth=0.1,
        edgecolor="white",
        legend=True,
        ax=ax,
    )

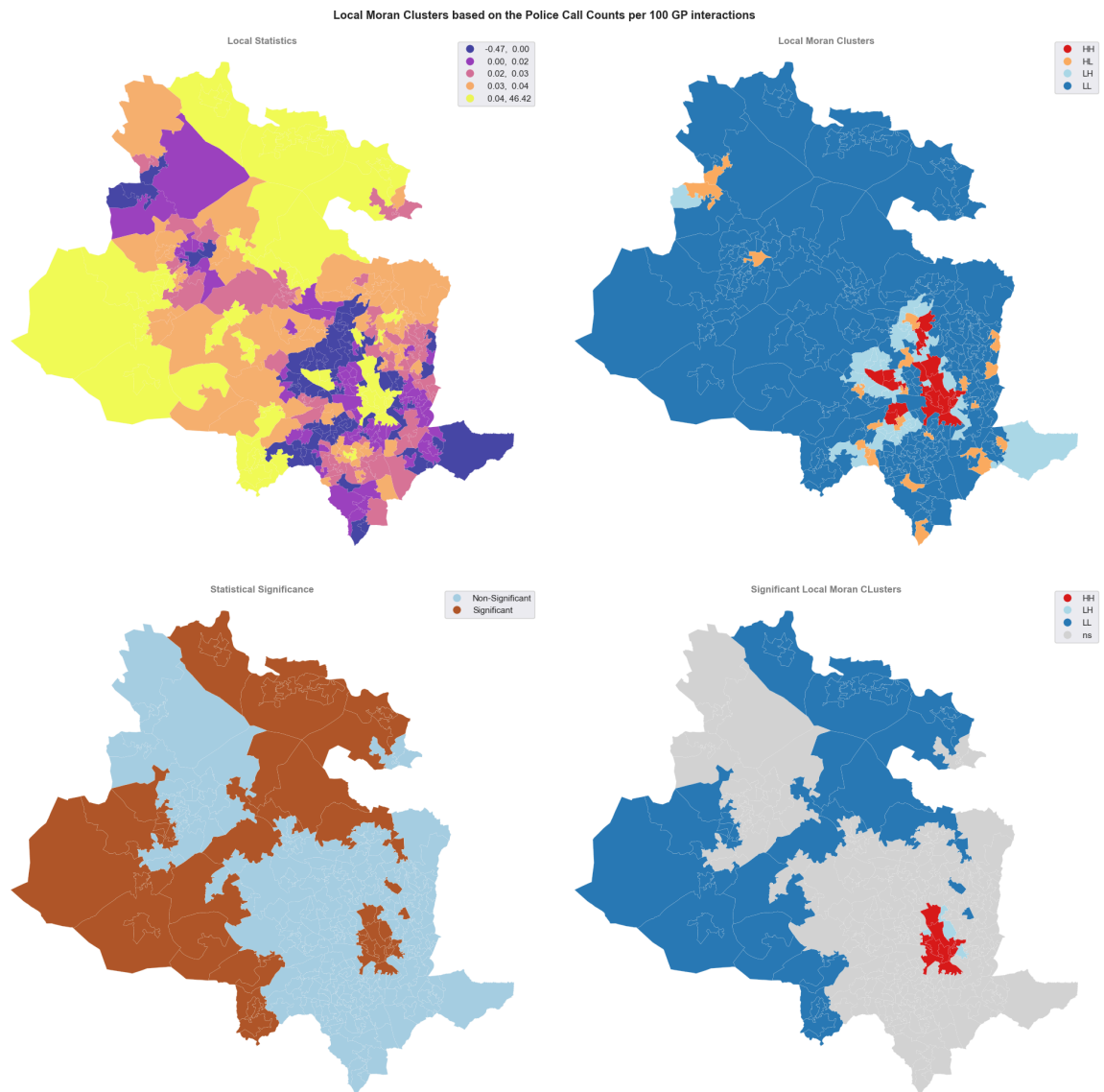
    # Cluster map
    ax = axs[3]
    # use a 5% significance level to select polygons as part of statistically signif
    esdplot.lisa_cluster(lisa_gpc_est, gdf_residual_yearwise_mdcrs, p=0.05, ax=ax)

    # Set title to each subplot
    for i, ax in enumerate(axs.flatten()):
        ax.set_axis_off()
        ax.set_title(
            [
                "Local Statistics",
                "Local Moran Clusters",
                "Statistical Significance",
                "Significant Local Moran Clusters",
            ][i],
            y=0.98,
            color='gray',
            fontweight='bold'
        )

    plt.suptitle('Local Moran Clusters based on the Police Call Counts per 100 GP in
f.tight_layout()

plt.show()

```



```
In [60]: # w = weights.distance.KNN.from_dataframe(gdf_residual_yearwise, k=4)
w = weights.Queen.from_dataframe(gdf_residual_yearwise)
w.transform = "R"

lisa = esda.moran.Moran_Local(gdf_residual_yearwise["olsresid"], w)

gdf_residual_yearwise_wgs84 = gdf_residual_yearwise.to_crs(epsg=3857)

f, axs = plt.subplots(nrows=2, ncols=2, figsize=(15, 15))
axs = axs.flatten()

# Choropleth of Local statistics
ax = axs[0]
gdf_residual_yearwise_wgs84.assign(
    Is=lisa.Is
    # Plot choropleth of Local statistics
).plot(
    column="Is",
    cmap="plasma",
    scheme="quantiles",
    k=5,
    edgecolor="white",
    linewidth=0.1,
    alpha=0.75,
```

```

        legend=True,
        ax=ax,
    )

    # Quadrant categories
    ax = axs[1]
    esdaplot.lisa_cluster(lisa, gdf_residual_yearwise_wgs84, p=1, ax=ax)

    # Significance map
    ax = axs[2]
    labels = pd.Series(
        1 * (lisa.p_sim < 0.05), # Assign 1 if significant, 0 otherwise
        index=gdf_residual_yearwise_wgs84.index
    ).map({1: "Significant", 0: "Non-Significant"})

    gdf_residual_yearwise_wgs84.assign(
        cl=labels
        # Plot choropleth of (non-)significant areas
    ).plot(
        column="cl",
        categorical=True,
        k=2,
        cmap="Paired",
        linewidth=0.1,
        edgecolor="white",
        legend=True,
        ax=ax,
    )

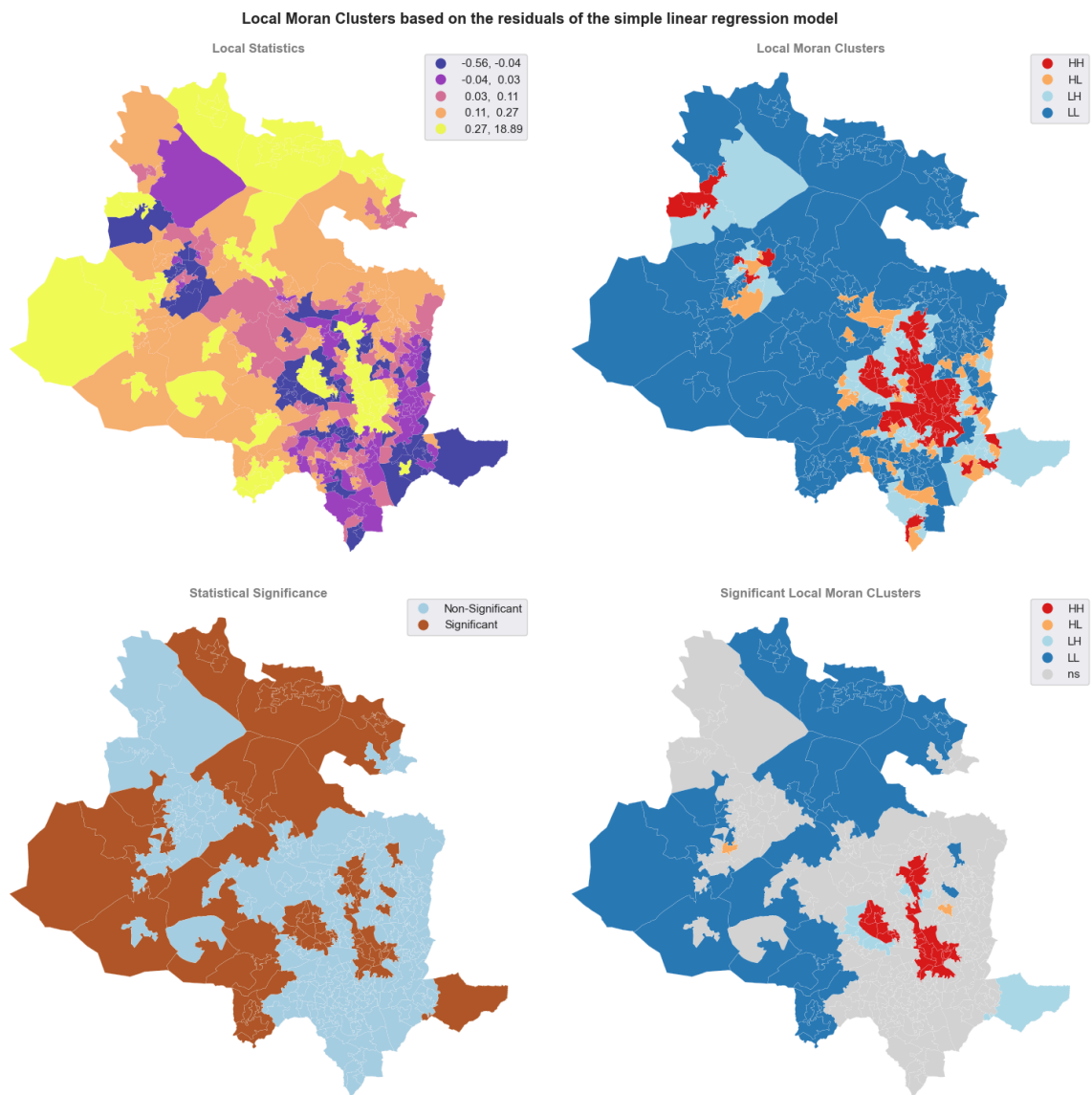
    # Cluster map
    ax = axs[3]
    # use a 5% significance level to select polygons as part of statistically signif
    esdaplot.lisa_cluster(lisa, gdf_residual_yearwise_wgs84, p=0.05, ax=ax)

    # Set title to each subplot
    for i, ax in enumerate(axs.flatten()):
        ax.set_axis_off()
        ax.set_title(
            [
                "Local Statistics",
                "Local Moran Clusters",
                "Statistical Significance",
                "Significant Local Moran Clusters",
            ][i],
            y=0.98,
            color='gray',
            fontweight='bold'
        )

    plt.suptitle('Local Moran Clusters based on the residuals of the simple linear r
    f.tight_layout()

    plt.show()

```



```
In [61]: lisa_gpc_est = esda.moran.Moran_Local(gdf_residual_yearwise_mdcrs["Police_calls_
labels = pd.Series(
    1 * (lisa_gpc_est.p_sim < 0.05), # Assign 1 if significant, 0 otherwise
    index=gdf_residual_yearwise_mdcrs.index
).map({1: "Significant", 0: "Non-Significant"}))

gdf_residual_yearwise_mdcrs = gdf_residual_yearwise_mdcrs.assign(
    cl=labels)
gdf_residual_yearwise_mdcrs['Quadrant_nw'] = np.where(gdf_residual_yearwise_mdcr
gdf_residual_yearwise_mdcrs.drop(['LSOA11CD', 'geometry'], axis=1, inplace=True)
```

```
In [62]: df_pop_lsoa = pd.read_csv('../data/Estimated_indicators_MH_Police.csv', index_
df_pop_21 = df_pop_lsoa[['LSOA11CD', 'Pop_21']]
```

```
In [63]: lsoa_cluster_pop = gdf_residual_yearwise_mdcrs.merge(df_pop_21[['LSOA11CD', 'Pop_
lsoa_lisa_cluster = lsoa_cluster_pop.merge(df_lsoa_bradford[['LSOA11CD', 'geometr
gdf_lsoa_lisa_cluster = gpd.GeoDataFrame(lsoa_lisa_cluster, geometry=lsoa_lisa_cl
gdf_lsoa_lisa_cluster_mdcrs = gdf_lsoa_lisa_cluster.to_crs(epsg=3857)

fig, ax = plt.subplots(figsize=(20,12))

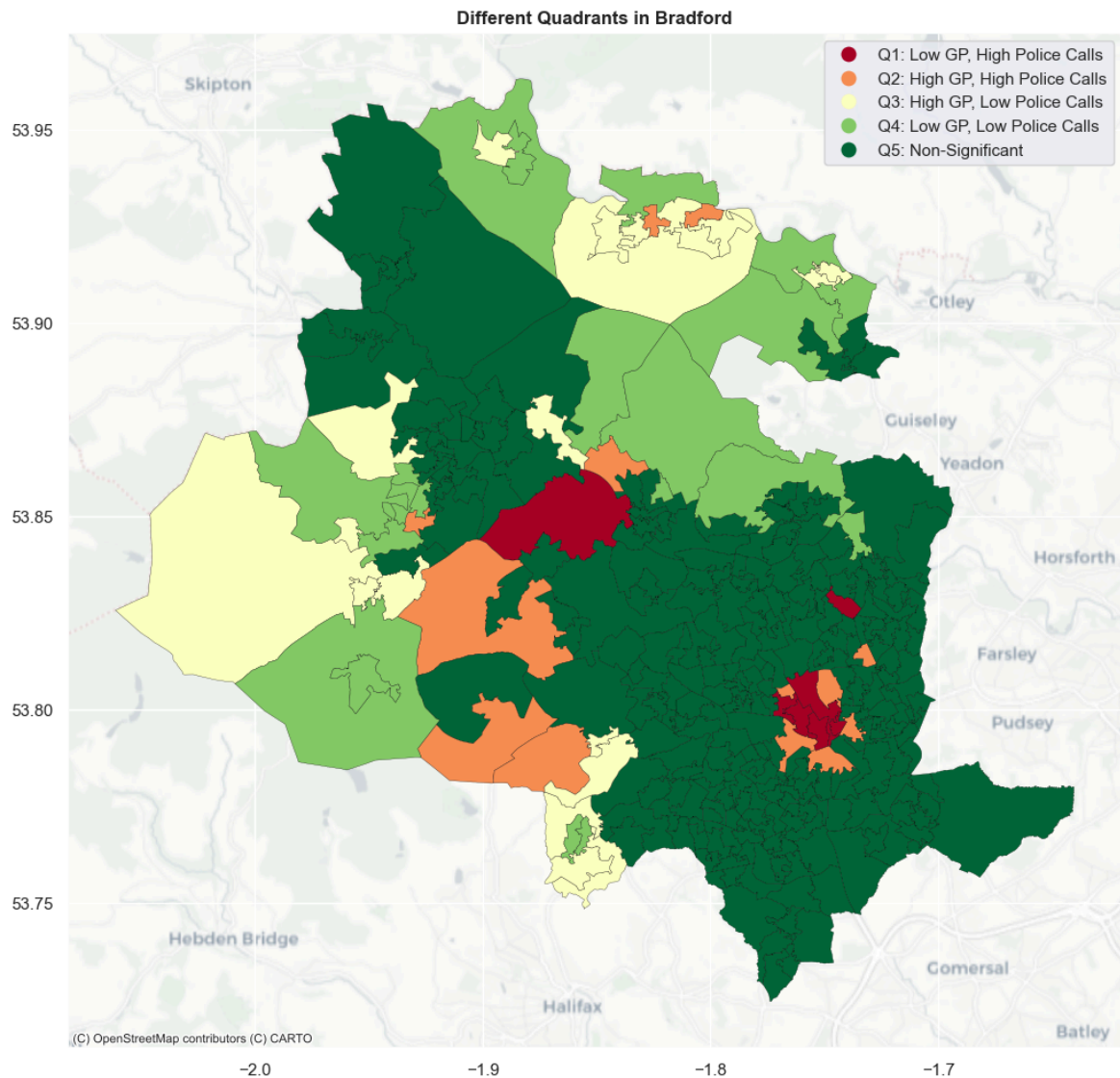
gdf_lsoa_lisa_cluster.plot(
    column='Quadrant_nw',
    cmap='RdYlGn',
```

```

        ax=ax,
        linewidth=0.2,
        edgecolor='black',
        legend=True
    )

    ax.set_title('Different Quadrants in Bradford', fontweight='bold')
    cx.add_basemap(ax, crs=gdf_lsoa_lisa_cluster.crs.to_string(), source=cx.provider

```



```

In [64]: f, axs = plt.subplots(nrows=3, ncols=2, figsize=(15, 20))
         axs = axs.flatten()

         # Choropleth of Local statistics
         ax = axs[0]
         gdf_lsoa_lisa_cluster_mdcrs.assign(
             Is=lisa_gpc_est.Is
         ).plot(
             column="Is",
             cmap="plasma",
             scheme="quantiles",
             k=5,
             edgecolor="white",
             linewidth=0.1,
             alpha=0.75,
             legend=True,

```



```

    ax=ax,
)

# Quadrant categories
ax = axs[1]
esdaplot.lisa_cluster(lisa_gpc_est, gdf_lsoa_lisa_cluster_mdcrs, p=1, ax=ax)

# Significance map
ax = axs[2]
labels = pd.Series(
    1 * (lisa_gpc_est.p_sim < 0.05), # Assign 1 if significant, 0 otherwise
    index=gdf_lsoa_lisa_cluster_mdcrs.index
).map({1: "Significant", 0: "Non-Significant"})

gdf_lsoa_lisa_cluster_mdcrs.assign(
    cl=labels
    # Plot choropleth of (non-)significant areas
).plot(
    column="cl",
    categorical=True,
    k=2,
    cmap="Paired",
    linewidth=0.1,
    edgecolor="white",
    legend=True,
    ax=ax,
)

# Cluster map
ax = axs[3]
# use a 5% significance level to select polygons as part of statistically signif
esdaplot.lisa_cluster(lisa_gpc_est, gdf_lsoa_lisa_cluster_mdcrs, p=0.05, ax=ax)

ax = axs[4]
gdf_lsoa_lisa_cluster_mdcrs.plot(
    column='Quadrant_nw',
    categorical=True,
    k=5,
    cmap="plasma",
    alpha=0.75,
    ax=ax,
    linewidth=0.2,
    edgecolor='white',
    legend=True,
    legend_kwds={
        'bbox_to_anchor': (0.75, 0.85),
        'loc': 'center left'
    }
)

ax = axs[5]
gdf_lsoa_lisa_cluster_mdcrs.plot(
    column='Pop_21',
    scheme="quantiles",
    k=5,
    cmap="plasma",
    alpha=0.75,
    ax=ax,
    linewidth=0.2,
    edgecolor='white',

```

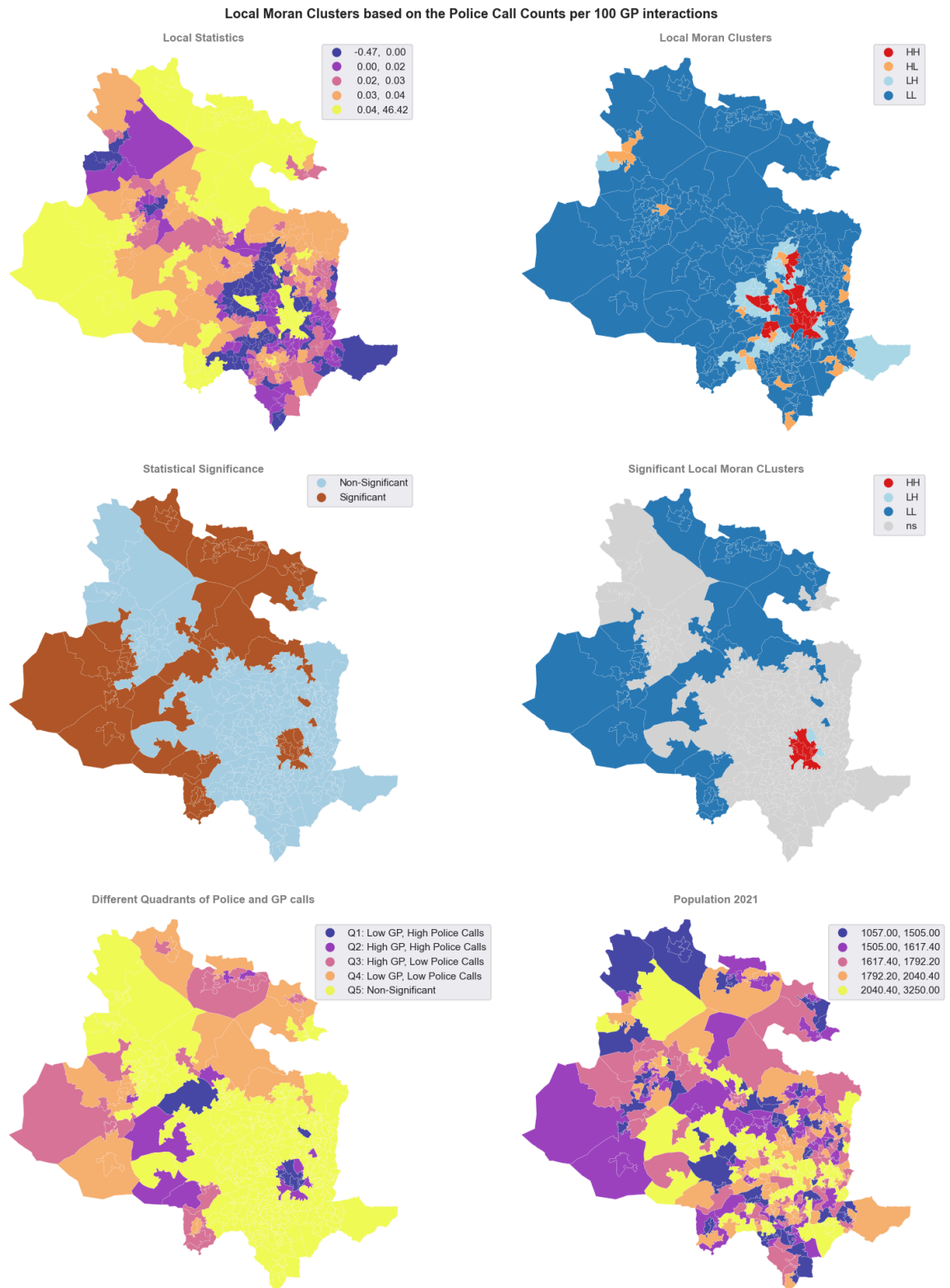
```
        legend=True,
        legend_kwds={
            'bbox_to_anchor': (0.75, 0.85),
            'loc': 'center left'
        }
    )

    # Set title to each subplot
    for i, ax in enumerate(axes.flatten()):
        ax.set_axis_off()
        ax.set_title(
            [
                "Local Statistics",
                "Local Moran Clusters",
                "Statistical Significance",
                "Significant Local Moran Clusters",
                "Different Quadrants of Police and GP calls",
                "Population 2021"
            ][i],
            y=0.98,
            color='gray',
            fontweight='bold'
        )
        # ax.legend(
        #     bbox_to_anchor=(0, 0)
        # )

plt.suptitle('Local Moran Clusters based on the Police Call Counts per 100 GP in

# f.delaxes(axes.flatten()[-1])
f.tight_layout()

plt.show()
```



```
In [65]: w_lisa = weights.Queen.from_dataframe(gdf_lsoa_lisa_cluster_mdcrs)
w_lisa.transform = "R"

std_df_lsoa = (gdf_lsoa_lisa_cluster_mdcrs['Police_calls_per_100_GP'] - gdf_lsoa
gdf_lsoa_lisa_cluster_mdcrs['w_Police_calls_per_100_GP'] = pd.Series(
    weights.lag_spatial(w_lisa, std_df_lsoa), index=std_df_lsoa.index
)
```

```
In [66]: lisa_gpc_est = esda.moran.Moran_Local(std_df_lsoa, w_lisa)
labels = pd.Series(
    1 * (lisa_gpc_est.p_sim < 0.05), # Assign 1 if significant, 0 otherwise
)
```

```

        index=gdf_lsoa_lisa_cluster_mdcrs.index
    ).map({1: "Significant", 0: "Non-Significant"})

gdf_lsoa_lisa_cluster_mdcrs= gdf_lsoa_lisa_cluster_mdcrs.assign(
    cl=labels)
gdf_lsoa_lisa_cluster_mdcrs['Quadrant_nw'] = np.where(gdf_lsoa_lisa_cluster_mdc
gdf_lsoa_lisa_cluster_mdcrs.drop(['LSOA11CD', 'geometry', 'Pop_21'],axis=1,inplace

```

```

In [67]: lsoa_cluster_pop = gdf_lsoa_lisa_cluster_mdcrs.merge(df_pop_21[['LSOA11CD', 'Pop_
lsoa_lisa_cluster = lsoa_cluster_pop.merge(df_lsoa_bradford[['LSOA11CD', 'geometr
gdf_lsoa_lisa_cluster = gpd.GeoDataFrame(lsoa_lisa_cluster, geometry=lsoa_lisa_cl
gdf_lsoa_lisa_cluster_mdcrs = gdf_lsoa_lisa_cluster.to_crs(epsg=3857)

f, axs = plt.subplots(nrows=3, ncols=2, figsize=(15, 20))
axs = axs.flatten()

# Choropleth of Local statistics
ax = axs[0]
gdf_lsoa_lisa_cluster_mdcrs.assign(
    Is=lisa_gpc_est.Is
    # Plot choropleth of Local statistics
).plot(
    column="Is",
    cmap="plasma",
    scheme="quantiles",
    k=5,
    edgecolor="white",
    linewidth=0.1,
    alpha=0.75,
    legend=True,
    ax=ax,
)

# Quadrant categories
ax = axs[1]
esdaplot.lisa_cluster(lisa_gpc_est, gdf_lsoa_lisa_cluster_mdcrs, p=1, ax=ax)

# Significance map
ax = axs[2]
labels = pd.Series(
    1 * (lisa_gpc_est.p_sim < 0.05), # Assign 1 if significant, 0 otherwise
    index=gdf_lsoa_lisa_cluster_mdcrs.index
).map({1: "Significant", 0: "Non-Significant"})

gdf_lsoa_lisa_cluster_mdcrs.assign(
    cl=labels
    # Plot choropleth of (non-)significant areas
).plot(
    column="cl",
    categorical=True,
    k=2,
    cmap="Paired",
    linewidth=0.1,
    edgecolor="white",
    legend=True,
    ax=ax,
)

# Cluster map
ax = axs[3]

```

```

# use a 5% significance level to select polygons as part of statistically signif
esdaplot.lisa_cluster(lisa_gpc_est, gdf_lsoa_lisa_cluster_mdcrs, p=0.05, ax=ax)

ax = axs[4]
gdf_lsoa_lisa_cluster_mdcrs.plot(
    column='Quadrant_nw',
    categorical=True,
    k=5,
    cmap="plasma",
    alpha=0.75,
    ax=ax,
    linewidth=0.2,
    edgecolor='white',
    legend=True,
    legend_kwds={
        'bbox_to_anchor': (0.75, 0.85),
        'loc': 'center left'
    }

)

ax = axs[5]
gdf_lsoa_lisa_cluster_mdcrs.plot(
    column='Pop_21',
    scheme="quantiles",
    k=5,
    cmap="plasma",
    alpha=0.75,
    ax=ax,
    linewidth=0.2,
    edgecolor='white',
    legend=True,
    legend_kwds={
        'bbox_to_anchor': (0.75, 0.85),
        'loc': 'center left'
    }

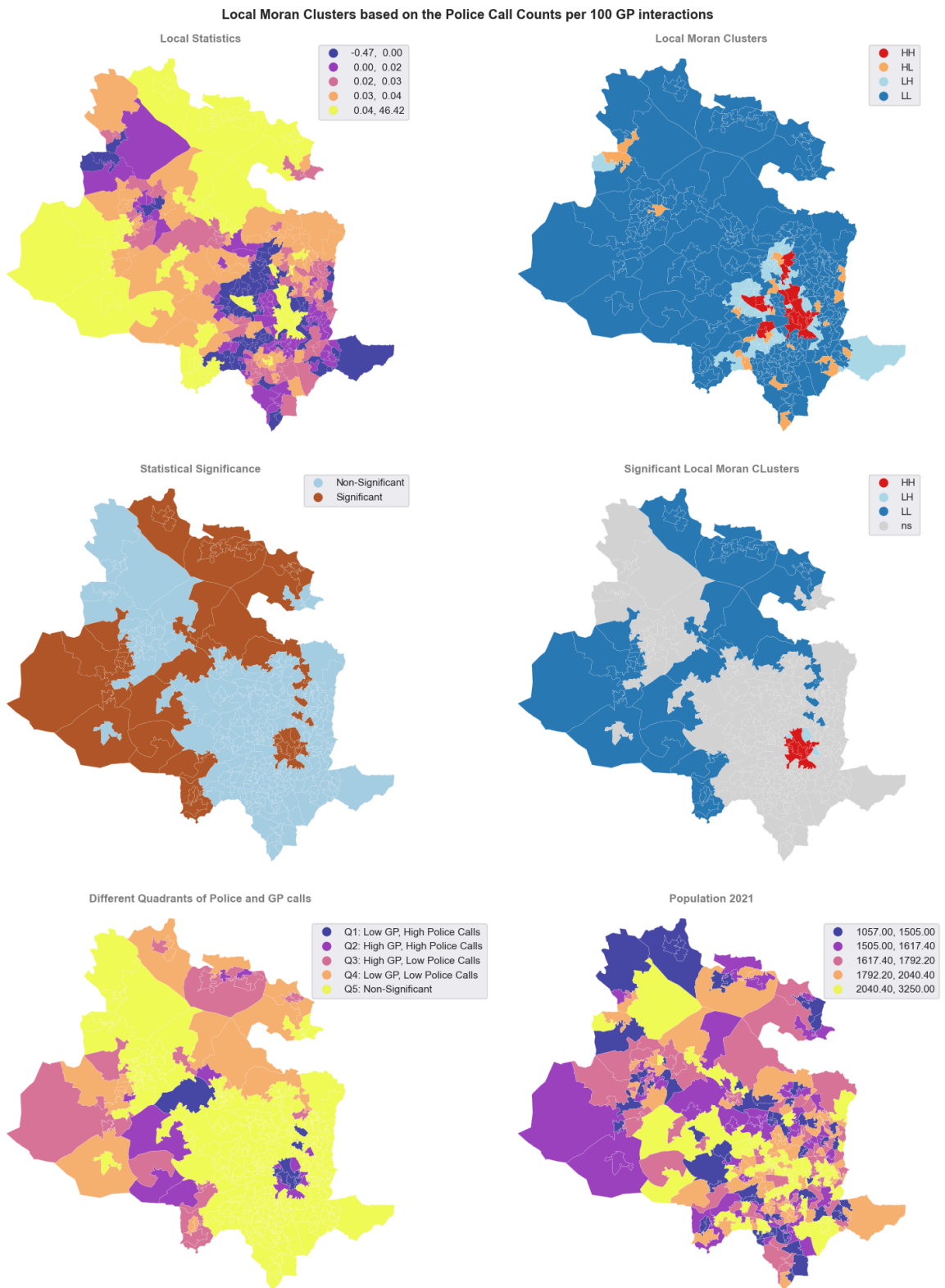
)

# Set title to each subplot
for i, ax in enumerate(axs.flatten()):
    ax.set_axis_off()
    ax.set_title(
        [
            "Local Statistics",
            "Local Moran Clusters",
            "Statistical Significance",
            "Significant Local Moran Clusters",
            "Different Quadrants of Police and GP calls",
            "Population 2021"
        ][i],
        y=0.98,
        color='gray',
        fontweight='bold'
    )
    # ax.legend(
    #     bbox_to_anchor=(0, 0)
    # )

plt.suptitle('Local Moran Clusters based on the Police Call Counts per 100 GP in
f.tight_layout()

```

```
plt.show()
```



GP Practices Shapefile

```
In [68]: gp_practices= pd.read_csv('.../data/gp_practices_by_constituency.csv',index_co
gdf_gp_practices = gpd.GeoDataFrame(gp_practices,geometry=gpd.points_from_xy(gp_
```

```
In [69]: gdf_gp_practices.crs
```



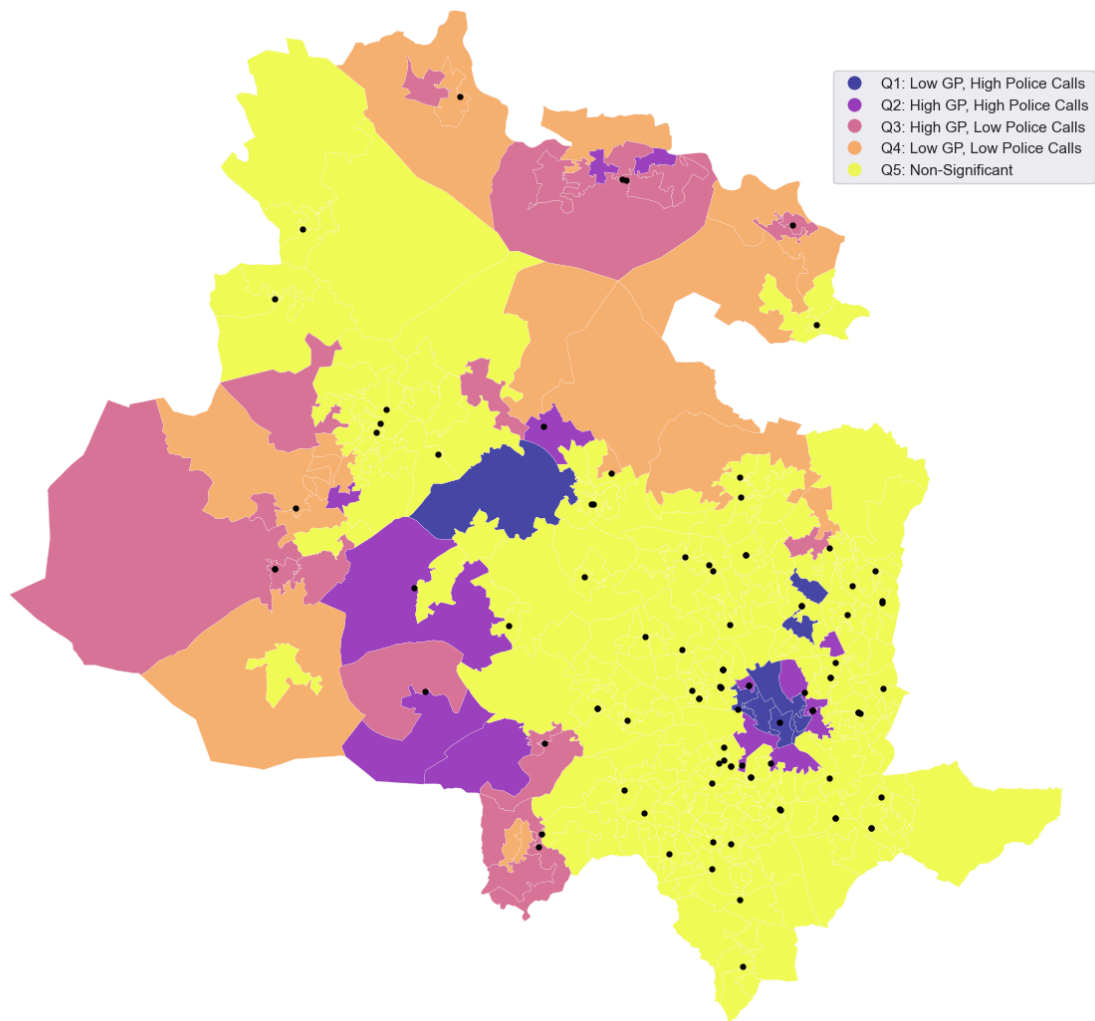
```
Out[69]: <Geographic 2D CRS: EPSG:4326>
Name: WGS 84
Axis Info [ellipsoidal]:
- Lat[north]: Geodetic latitude (degree)
- Lon[east]: Geodetic longitude (degree)
Area of Use:
- name: World.
- bounds: (-180.0, -90.0, 180.0, 90.0)
Datum: World Geodetic System 1984 ensemble
- Ellipsoid: WGS 84
- Prime Meridian: Greenwich
```

```
In [70]: bradford = gdf_lsoa_lisa_cluster_mdcrs.geometry.unary_union
gdf_gp_practices = gdf_gp_practices.to_crs(gdf_lsoa_lisa_cluster_mdcrs.crs)
gp_practices_bradford = gdf_gp_practices[gdf_gp_practices.geometry.within(bradfo
```

```
In [71]: f, ax = plt.subplots(figsize=(15, 20))

gdf_lsoa_lisa_cluster_mdcrs.plot(
    column='Quadrant_nw',
    categorical=True,
    k=5,
    cmap="plasma",
    alpha=0.75,
    ax=ax,
    linewidth=0.2,
    edgecolor='white',
    legend=True,
    legend_kwds={
        'bbox_to_anchor': (0.75, 0.85),
        'loc': 'center left'
    }
)

gp_practices_bradford.plot(ax=ax, color='black', markersize=12)
ax.set_axis_off()
```



```
In [72]: gdf_lsoa_lisa_cluster_mdcrs.rename(columns={'Call Counts':'Call_counts'}, inplace=True)
```

```
In [73]: import matplotlib.gridspec as gridspec
import matplotlib.patches as mpatches

sns.set_style('ticks')

fig = plt.figure(figsize=(15, 6))
gs = gridspec.GridSpec(1, 3, width_ratios=[1, 1.5, 1.5]) # Give more space to the third plot

ax1 = fig.add_subplot(gs[0])
ax2 = fig.add_subplot(gs[1])
ax3 = fig.add_subplot(gs[2])

# Calculate means
median_x = np.median(gdf_lsoa_lisa_cluster_mdcrs['Call_counts'])
median_y = np.median(gdf_lsoa_lisa_cluster_mdcrs['Police Call Counts'])

x = gdf_lsoa_lisa_cluster_mdcrs['Call_counts']
y = gdf_lsoa_lisa_cluster_mdcrs['Police Call Counts']

quadrants = gdf_lsoa_lisa_cluster_mdcrs['Quadrant'].unique()
colors = {}
for xi, yi in zip(x, y):
    if xi >= median_x and yi >= median_y:
```

```

        colors['Q2: High GP, High Police Calls']='#E69F00'      # High-High (HH)
    elif xi < median_x and yi < median_y:
        colors['Q4: Low GP, Low Police Calls']='#009E73'      # Low-Low (LL)
    elif xi >= median_x and yi < median_y:
        colors['Q3: High GP, Low Police Calls']='#0072B2'     # High-Low (HL)
    else:
        colors['Q1: Low GP, High Police Calls']='#D55E00'     # Low-High (LH)

## First Plot *****
# Create scatter plot
sns.scatterplot(data=gdf_lsoa_lisa_cluster_mdcrs,x='Call_counts', y='Police Call

# Add mean Lines to represent axes centered at the mean
ax1.axvline(median_x, color='black', linestyle='--')
ax1.axhline(median_y, color='black', linestyle='--')

b, a = np.polyfit(x, y, 1)
sns.lineplot(x=x, y=a + b * x,ax=ax1, color='black', linestyle='-', linewidth=2)

x_range = max(x) - min(x)
y_range = max(y) - min(y)
ax1.set_xlim(10, median_x + x_range/2)
ax1.set_ylim(-50, median_y + y_range/2)

# Labeling and title

ax1.text(median_x + x_range/6, median_y + y_range/4, 'High-High', fontsize=12,fc
ax1.text(median_x - x_range/4, median_y - y_range/10, 'Low-Low', fontsize=12,for
ax1.text(median_x + x_range/6, median_y - y_range/10, 'High-Low', fontsize=12,fc
ax1.text(median_x - x_range/4, median_y + y_range/4, 'Low-High', fontsize=12, fc
ax1.set_xlabel('Mental Health related GP interactions',fontweight='bold',fontsiz
ax1.set_ylabel('Police Call Counts related to Mental Health',fontweight='bold',f
# plt.legend(bbox_to_anchor=(1,1.02))
ax1.grid(True)
sns.despine()

## Second Plot *****
# Spatial distribution of quadrants
gdf_lsoa_lisa_cluster_mdcrs.plot(
    column='Quadrant',
    color=gdf_lsoa_lisa_cluster_mdcrs['Quadrant'].map(colors),
    ax=ax2,
    linewidth=0.2,
    edgecolor='white',
    legend=True,alpha=0.8
)

quadrant_colors = {
    'Low-High': '#D55E00',
    'High-High': '#E69F00',
    'High-Low': '#0072B2',
    'Low-Low': '#009E73'
}

legend_patches = [mpatches.Patch(color=color, label=label) for label, color in q
ax2.legend(handles=legend_patches, title='Quadrants', loc='lower left',bbox_to_a

ax2.set_title('Different Quadrants in Bradford',fontweight='bold')
ax2.set_axis_off()
# cx.add_basemap(ax1, crs=gdf_lsoa_lisa_cluster_mdcrs.crs.to_string(), source=cx

```

```

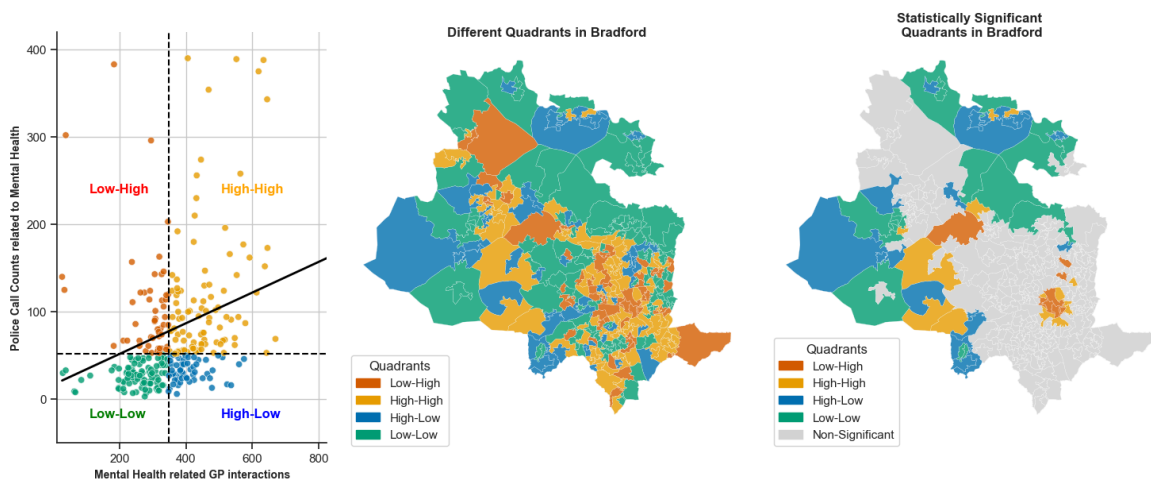
## Third Plot *****
# Spatial distribution of statistically significant quadrants
# Update the color dict to contain the non-significant color
colors['Q5: Non-Significant'] = '#D3D3D3'
quadrant_colors['Non-Significant'] = '#D3D3D3'

gdf_lsoa_lisa_cluster_mdcrs.plot(
    column='Quadrant_nw',
    color=gdf_lsoa_lisa_cluster_mdcrs['Quadrant_nw'].map(colors),
    ax=ax3,
    linewidth=0.2,
    edgecolor='white',
    legend=True, alpha=0.8
)

legend_patches = [mpatches.Patch(color=color, label=label) for label, color in quadrant_colors.items()]
ax3.legend(handles=legend_patches, title='Quadrants', loc='lower left', bbox_to_anchor=(0.5, 0.1))
ax3.set_title('Statistically Significant\n Quadrants in Bradford', fontweight='bold')
ax3.set_axis_off()
# cx.add_basemap(ax2, crs=gdf_lsoa_lisa_cluster_mdcrs.crs.to_string(), source=cx)

plt.tight_layout()
plt.show()

```



Regression Analysis

- Demographics: Age, ethnicity, Population density
- Socioeconomic status: Unemployment rate, Index of Multiple Deprivation (IMD)
- Urban infrastructure: Road Density, availability of safe public spaces, public transport access

```

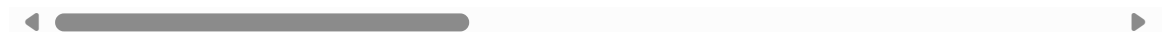
In [74]: predictors_df = pd.read_csv('../data/predictors_bradford_data.csv', index_col=0)
predictors_df[~predictors_df['mnemonic'].isin(df_lsoa_bradford['LSOA11CD'])]

```

Out[74]:

	mnemonic	Total_pop	Female	Male	Total_HHs	1_HH	2_HH	3_HH	4_HH
2021 super output area - lower layer	Bradford 030E	E01033898	1594	811	783	456	73	97	60
	Bradford 030F	E01033899	1651	894	757	475	45	89	126
	Bradford 044G	E01033900	1584	770	814	407	74	58	52
	Bradford 044H	E01033901	1666	844	822	407	45	64	64

4 rows × 21 columns



```

In [75]: new_row_1 = predictors_df.loc['Bradford 030E']+predictors_df.loc['Bradford 030F']
new_row_2 = predictors_df.loc['Bradford 044G']+predictors_df.loc['Bradford 044H']

mod_predictors_df = predictors_df.copy()
mod_predictors_df_1 = mod_predictors_df.drop(['Bradford 030E','Bradford 030F','B

mod_predictors_df_1.loc['Bradford 030C']=new_row_1
mod_predictors_df_1.loc['Bradford 044B']=new_row_2

In [76]: mod_predictors_df_1.loc['Bradford 030C',['mnemonic','IMD_score']] = ['E01010677'
mod_predictors_df_1.loc['Bradford 044B',['mnemonic','IMD_score']] = ['E01010835'

In [77]: mod_predictors_df_1.reset_index(inplace=True)

In [78]: gdf_lsoa_lisa_cluster_mdcrs['area_hec'] = gdf_lsoa_lisa_cluster_mdcrs['geometry']

In [79]: merged_predictors_lsoa = gdf_lsoa_lisa_cluster_mdcrs[['Assigned_LSOA','Police Ca

In [80]: merged_predictors_lsoa['Pop_density'] = merged_predictors_lsoa['Total_pop']/merg
merged_predictors_lsoa['1_2_HH'] = merged_predictors_lsoa['1_HH'] +merged_predic
merged_predictors_lsoa['3_more_HH'] = merged_predictors_lsoa['3_HH'] +merged_pre
merged_predictors_lsoa['Age_50_over'] = merged_predictors_lsoa['Age_50_64'] +mer

In [81]: merged_predictors_lsoa.drop(['2021 super output area - lower layer', 'mnemonic',

In [82]: merged_predictors_lsoa[['Police Call Counts','Call_counts','Female', 'Male','Age
'Black, Black British, Black Welsh, Caribbean or African',

```

```

'Mixed or Multiple ethnic groups', 'White', 'Other ethnic group',
'Unemployment']] = merged_predictors_lsoa[['Police Call Counts', 'Call_cou
'Black, Black British, Black W
'Mixed or Multiple ethnic grou
'Unemployment']].div(merged_pr

merged_predictors_lsoa[['1_2_HH', '3_more_HH']] = merged_predictors_lsoa[['1_2_HH

# merged_predictors_lsoa[['Police Call Counts', 'Call_counts']] = (merged_predict

```

```

In [83]: merged_predictors_lsoa.rename(columns={'Police Call Counts': 'Police_Call_Counts',
'Black, Black British, Black Welsh, Caribbean or
'Mixed or Multiple ethnic groups': 'Mixed_or_Mult
'Other ethnic group': 'Other', 'Age_25_34': 'Age_2
# '1_HH': 'HH_1', '2_HH': 'HH_2', '3_HH': 'HH_3', '4_
'1_2_HH': 'HH_1_2', '3_more_HH': 'HH_3_more'}, inplace=

```

```

In [84]: merged_predictors_lsoa.columns = [col.replace('_', '') for col in merged_predicto

```

```

In [85]: independent_var = merged_predictors_lsoa.columns.drop(['AssignedLSOA', 'PoliceCa
'Age24below', 'White', 'HH12
independent_var

```

```

Out[85]: Index(['Callcounts', 'Female', 'Age2534', 'Age3549', 'Asian', 'Black',
'MixedorMultiple', 'Other', 'Unemployment', 'IMDscore', 'Popdensity',
'HH3more', 'Age50over'],
dtype='object')

```

```

In [86]: formula = f"PoliceCallCounts ~ {' + '.join(independent_var)}"
formula

```

```

Out[86]: 'PoliceCallCounts ~ Callcounts + Female + Age2534 + Age3549 + Asian + Black + M
ixedorMultiple + Other + Unemployment + IMDscore + Popdensity + HH3more + Age50
over'

```

```

In [87]: from statsmodels.stats.outliers_influence import variance_inflation_factor
import statsmodels.formula.api as smf

predictors_data = merged_predictors_lsoa[independent_var]
vif_data = pd.DataFrame()
vif_data["Feature"] = independent_var
vif_data["VIF"] = [variance_inflation_factor(predictors_data.values, i) for i in
# VIF
print("Variance Inflation Factors:")
print(vif_data)

```


Variance Inflation Factors:

	Feature	VIF
0	Callcounts	16.257229
1	Female	458.608065
2	Age2534	34.195095
3	Age3549	116.730766
4	Asian	23.540015
5	Black	5.551770
6	MixedorMultiple	9.318102
7	Other	6.152795
8	Unemployment	188.419310
9	IMDscore	21.112176
10	Popdensity	5.351274
11	HH3more	62.004304
12	Age50over	124.951148

```
In [88]: from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()
scaled_predictors_data = pd.DataFrame(scaler.fit_transform(predictors_data.iloc[
scaled_predictors_data_df = pd.concat([predictors_data.iloc[:,0],scaled_predicto

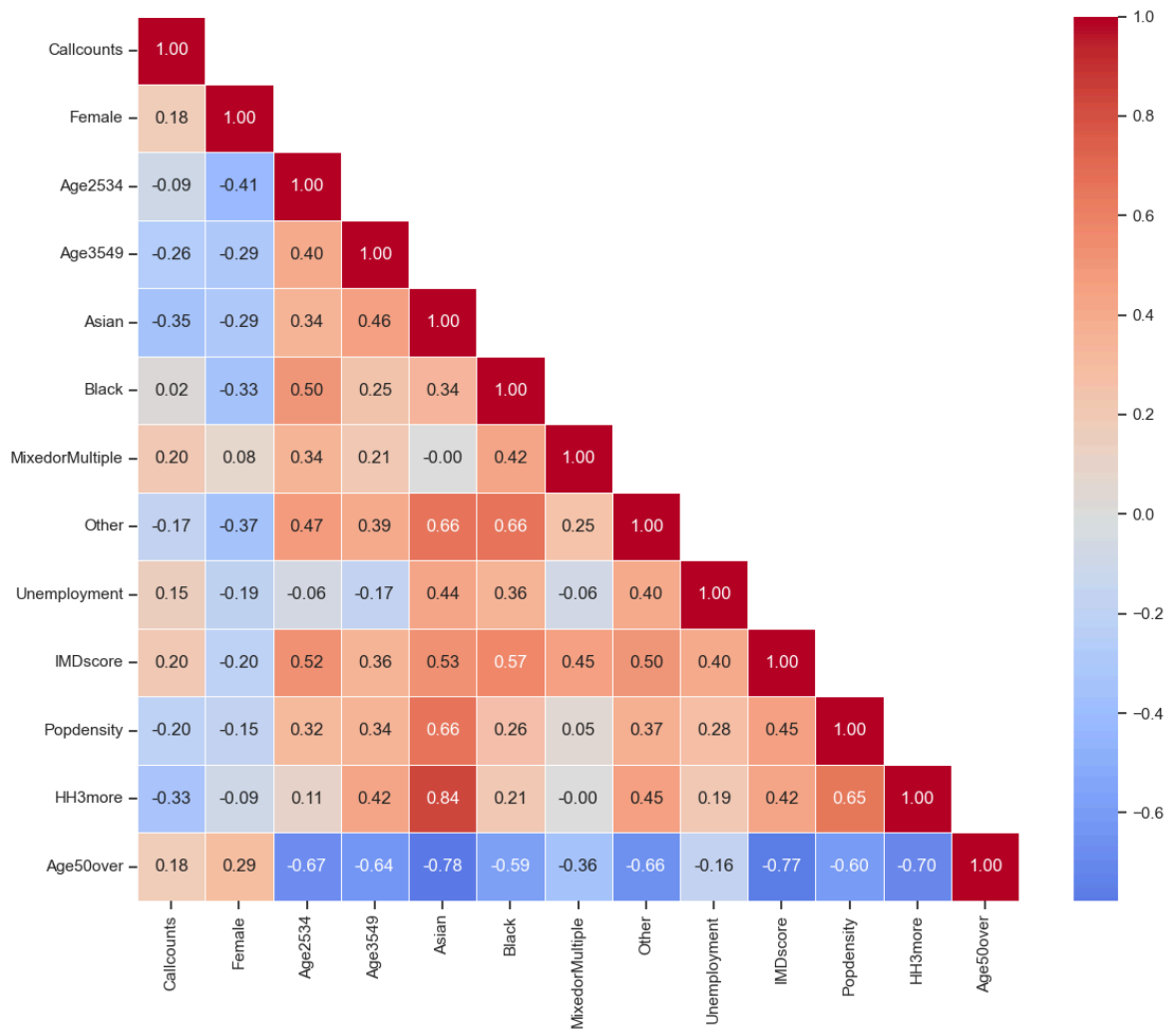
vif_data = pd.DataFrame()
vif_data["Feature"] = independent_var
vif_data["VIF"] = [variance_inflation_factor(scaled_predictors_data_df.values, i
# VIF
print("Variance Inflation Factors:")
print(vif_data)
```

Variance Inflation Factors:

	Feature	VIF
0	Callcounts	1.045647
1	Female	1.573067
2	Age2534	4.864378
3	Age3549	2.429674
4	Asian	11.115688
5	Black	3.213006
6	MixedorMultiple	1.778549
7	Other	3.227961
8	Unemployment	3.959204
9	IMDscore	4.775150
10	Popdensity	2.170602
11	HH3more	8.112262
12	Age50over	20.904471

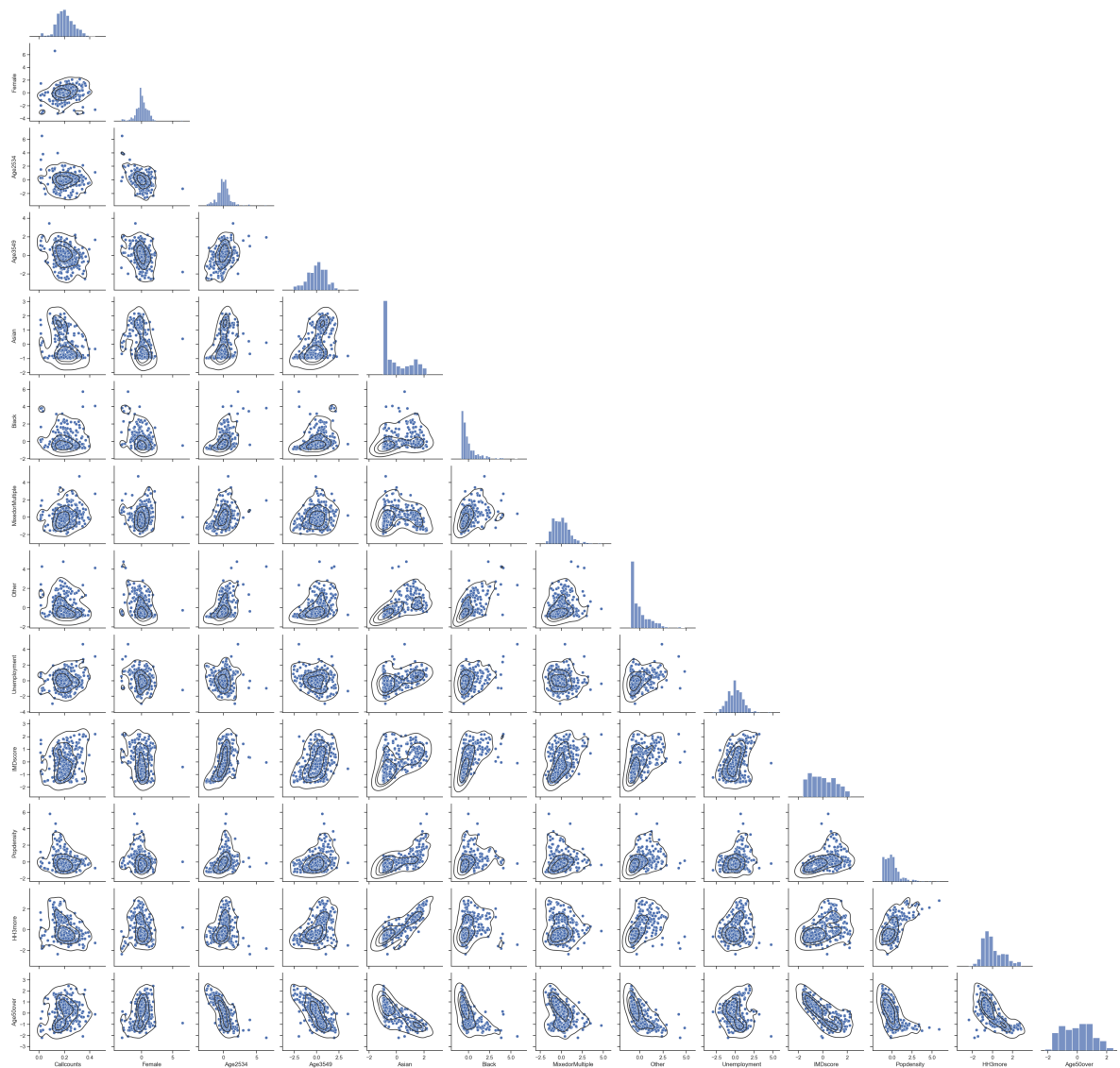
```
In [89]: correlation_matrix = predictors_data.corr()
mask = np.triu(correlation_matrix,1)

plt.figure(figsize=(12, 10))
sns.heatmap(correlation_matrix,mask=mask, annot=True, cmap='coolwarm', center=0,
plt.tight_layout()
plt.show()
```



```
In [90]: g = sns.pairplot(pd.DataFrame(scaled_predictors_data_df, columns=independent_var))
g.map_lower(sns.kdeplot, levels=4, color=".2")
```

```
Out[90]: <seaborn.axisgrid.PairGrid at 0x12a59ce7a10>
```



```
In [91]: scaled_df = pd.concat([merged_predictors_lsoa['PoliceCallCounts'], scaled_predict
```

```
In [92]: m1 = smf.ols(formula=formula, data=scaled_df).fit()
m1.summary()
```

Out[92]:

OLS Regression Results

Dep. Variable:	PoliceCallCounts	R-squared:	0.425
Model:	OLS	Adj. R-squared:	0.399
Method:	Least Squares	F-statistic:	16.81
Date:	Thu, 21 Aug 2025	Prob (F-statistic):	9.41e-29
Time:	12:12:46	Log-Likelihood:	579.86
No. Observations:	310	AIC:	-1132.
Df Residuals:	296	BIC:	-1079.
Df Model:	13		
Covariance Type:	nonrobust		

	coef	std err	t	P> t	[0.025	0.975]
Intercept	0.0343	0.009	4.017	0.000	0.018	0.051
Callcounts	0.0449	0.040	1.111	0.267	-0.035	0.124
Female	-0.0021	0.003	-0.785	0.433	-0.007	0.003
Age2534	0.0184	0.005	3.837	0.000	0.009	0.028
Age3549	0.0118	0.003	3.493	0.001	0.005	0.018
Asian	-0.0128	0.007	-1.705	0.089	-0.027	0.002
Black	-0.0047	0.004	-1.207	0.229	-0.012	0.003
MixedorMultiple	0.0009	0.003	0.322	0.748	-0.005	0.007
Other	0.0023	0.004	0.584	0.560	-0.005	0.010
Unemployment	0.0209	0.004	4.778	0.000	0.012	0.030
IMDscore	-0.0035	0.005	-0.702	0.483	-0.013	0.006
Popdensity	-0.0117	0.003	-3.668	0.000	-0.018	-0.005
HH3more	-0.0073	0.006	-1.177	0.240	-0.019	0.005
Age50over	-0.0202	0.010	-2.031	0.043	-0.040	-0.001

Omnibus:	287.182	Durbin-Watson:	1.714
Prob(Omnibus):	0.000	Jarque-Bera (JB):	8404.983
Skew:	3.745	Prob(JB):	0.00
Kurtosis:	27.385	Cond. No.	44.5

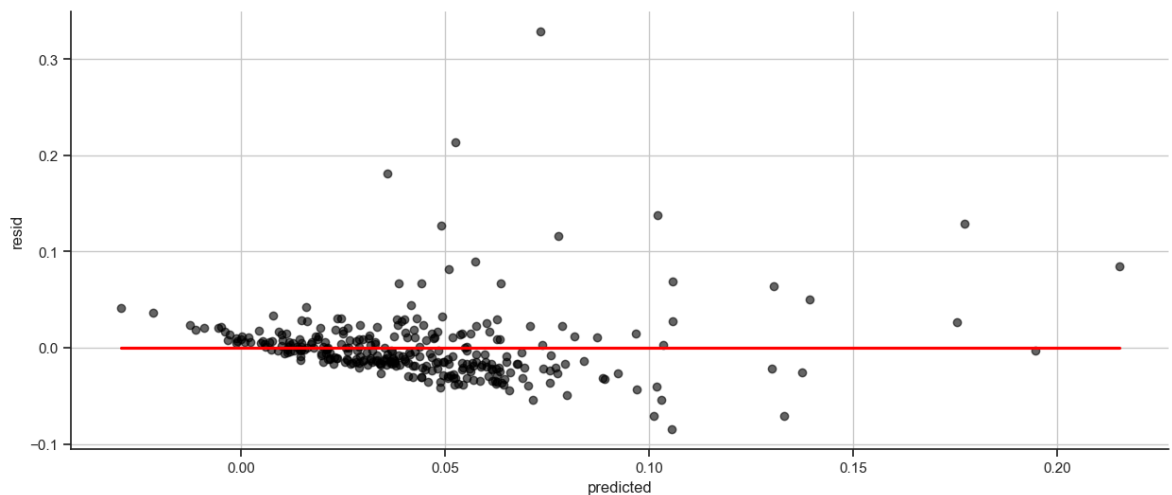
Notes:

[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.

```
In [93]: scaled_df['resid'] = m1.resid
scaled_df['predicted'] = m1.predict()

sns.set_style('ticks')
fig,ax = plt.subplots(figsize=(15, 6))

sns.regplot(
    x='predicted', y='resid', data=scaled_df, ci=None,ax=ax, color='red',scatter
)
sns.despine()
ax.grid()
```

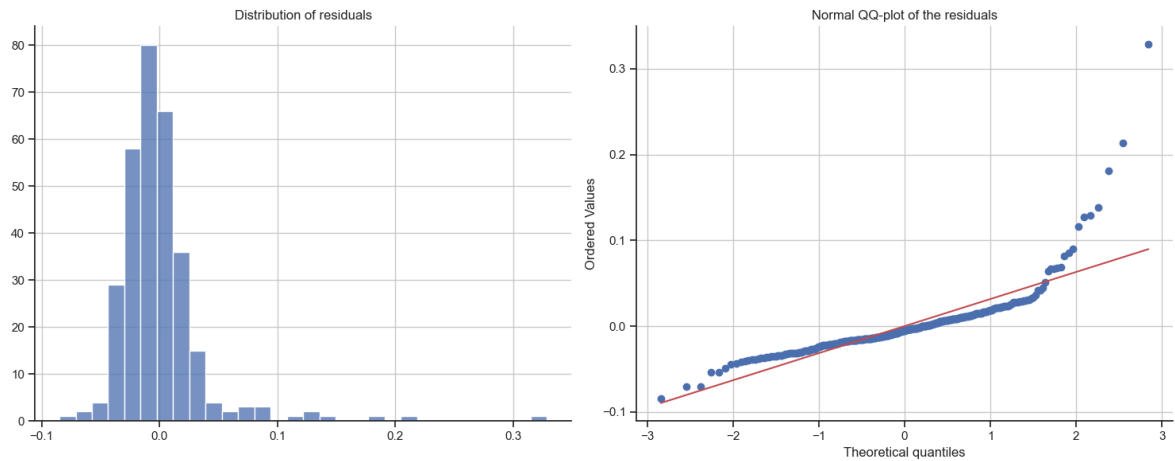


```
In [94]: import scipy.stats as stats

fig,ax = plt.subplots(1,2,figsize=(15, 6))
ax1 = ax[0]
sns.histplot(scaled_df['resid'],ax=ax1, kde=False, bins=30)
ax1.set_title("Distribution of residuals")
ax1.set_xlabel("")
ax1.set_ylabel("")
ax1.grid()
sns.despine()

ax2 =ax[1]
stats.probplot(scaled_df['resid'], dist="norm", plot=ax2)
ax2.set_title("Normal QQ-plot of the residuals")
ax2.grid()
sns.despine()

plt.tight_layout()
plt.show()
```

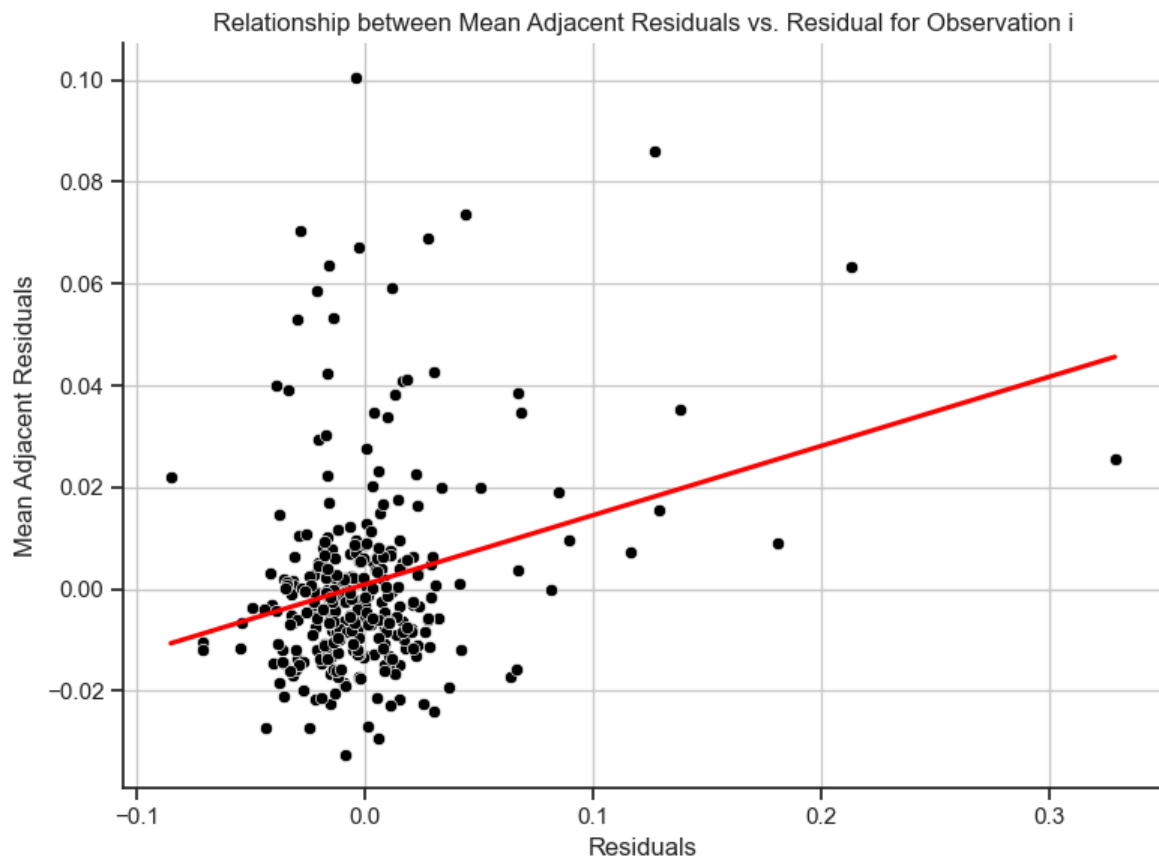


```
In [95]: from libpysal.weights import Queen

scaled_df_m1lsoa = pd.concat([merged_predictors_lsoa['AssignedLSOA'], scaled_df],
scaled_df_m1lsoa = scaled_df_m1lsoa.merge(df_lsoa_bradford[['LSOA11CD', 'geometry'],
gdf_scaled_m1lsoa = gpd.GeoDataFrame(scaled_df_m1lsoa, geometry=scaled_df_m1lsoa[
w = Queen.from_dataframe(gdf_scaled_m1lsoa)

# Calculate average neighboring residuals
residuals = gdf_scaled_m1lsoa['resid'].values
resnb_calc = [residuals[neighbors].mean() if neighbors else None for neighbors i
gdf_scaled_m1lsoa['resnb'] = resnb_calc

# Plot residuals vs. mean adjacent residuals
plt.figure(figsize=(8, 6))
sns.scatterplot(data=gdf_scaled_m1lsoa, x='resid', y='resnb', color='black')
sns.regplot(data=gdf_scaled_m1lsoa, x='resid', y='resnb', scatter=False, ci=None)
plt.xlabel("Residuals")
plt.ylabel("Mean Adjacent Residuals")
plt.title("Relationship between Mean Adjacent Residuals vs. Residual for Observa
sns.despine()
plt.grid()
plt.tight_layout()
plt.show()
```

```
In [96]: from esda.moran import Moran

# W_binary = w.full()[0]

mi = Moran(gdf_scaled_mllsoa['resid'], w, permutations=1000)

print("Observed Moran's I:", mi.I)
print("Expected under null:", mi.EI)
print("p-value (permutations):", mi.p_sim)
print("z (permutation-based):", mi.z_sim)
```

```
Observed Moran's I: 0.13607579229821076
Expected under null: -0.003236245954692557
p-value (permutations): 0.001998001998001998
z (permutation-based): 4.009810396838955
```

Spatial Regression

```
In [97]: import statsmodels.formula.api as sm

f = (
    "PoliceCallCounts ~ "
    + " + ".join(independent_var)
    + " + Assigned_LSOA-1"
)

scaled_lsoa_df = pd.concat([gdf_lsoa_lisa_cluster_mdcrs['Assigned_LSOA'],merged_
m2 = smf.ols(f, data=scaled_lsoa_df).fit()

lsoa_fe = [i for i in m2.params.index if "Assigned_LSOA[" in i]
# Result summary
pd.DataFrame(
```

```

{
    "Coef.": m2.params[lsoa_fe],
    "Std. Error": m2.bse[lsoa_fe],
    "P-Value": m2.pvalues[lsoa_fe],
}
)

```

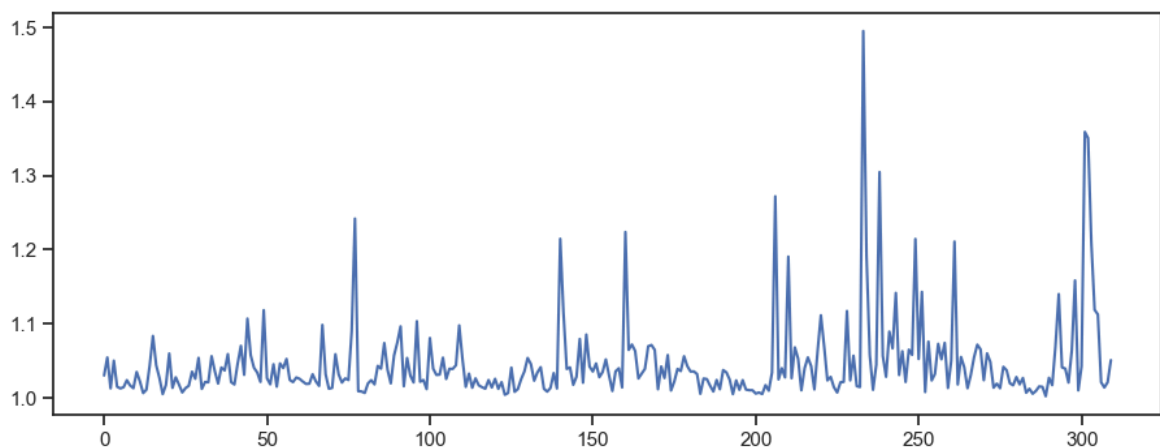
Out[97]:

	Coef.	Std. Error	P-Value
Assigned_LSOA[E01010568]	0.010717	inf	NaN
Assigned_LSOA[E01010569]	-0.003279	inf	NaN
Assigned_LSOA[E01010570]	0.025632	inf	NaN
Assigned_LSOA[E01010571]	-0.015835	inf	NaN
Assigned_LSOA[E01010572]	0.019705	inf	NaN
...	...	...	...
Assigned_LSOA[E01033693]	0.035685	inf	NaN
Assigned_LSOA[E01033694]	-0.065951	inf	NaN
Assigned_LSOA[E01033695]	0.023793	inf	NaN
Assigned_LSOA[E01033696]	0.008103	inf	NaN
Assigned_LSOA[E01033697]	0.037408	inf	NaN

310 rows × 3 columns

In [98]: `np.exp(merged_predictors_lsoa['PoliceCallCounts']).plot()`

Out[98]: <Axes: >



In []: